

E-commerce system MVP design scheme and technical understanding

Name: Lawrence Zhou **Date:** June 2026

I. User Roles & Core Scenarios

1.1 Project Background

Product Objectives

With the popularization of online shopping, e-commerce platforms have become an important channel for consumers to purchase goods. In order to meet the needs of consumers to browse and purchase goods online, as well as merchants to manage their products and orders, the company plans to build a basic e-commerce system from scratch.

This system needs to meet the core needs of two types of users simultaneously:

Consumer demand

Consumers hope to quickly complete the product selection and purchase process through the platform, including:

1. Browse product list
2. View product details
3. Search and filter products
4. add to the cart
5. Submit an order
6. Online payment
7. Check order status
8. Track logistics information

The core issue that consumers are concerned about is:

- Is the product information clear and complete?
- Is the order placement process simple and smooth?
- Is the payment process stable and reliable?
- Is the order status transparent and traceable?

Therefore, the system needs to provide a complete and stable purchase process to help users complete transactions smoothly.

Merchant operation side needs

In addition to consumers, the platform also needs to support operations staff in completing daily business management tasks, including:

1. Product creation and maintenance
2. SKU Management
3. Inventory Management
4. Order Management
5. Shipment Management
6. After-sales service

The core issues that operations personnel focus on are:

- Can the products be quickly put on the shelves for sale?
- Is the inventory accurate and controllable?
- Can orders be processed efficiently?
- Is the shipping process smooth?

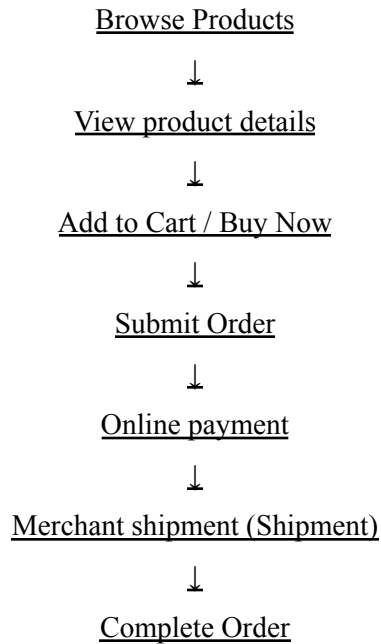
Therefore, the system needs to build a basic operational backend to support the sales of goods and the fulfillment of orders.

MVP range

This project is the first phase of product development (MVP, Minimum Viable Product), with the goal of verifying whether the e-commerce transaction loop can operate normally in the shortest possible time, rather than building a complete and mature e-commerce platform all at once.

Therefore, this phase only retains the functions necessary to support the core transaction process.

The core transaction process is as follows:



Through the above process, the system can complete a full business loop from product display to transaction completion.

MVP Construction Principles

To ensure the project can be launched quickly and its business value validated, this MVP design follows these principles:

1. Prioritize ensuring the availability of core transaction links.

Any features that affect the user's ability to complete the purchase process will be prioritized for development, including:

- Product Display
- SKU selection
- shopping cart
- Order creation
- Payment ability
- Shipment Management

Ensure that users can successfully complete a full purchase.

2. Prioritize ensuring data accuracy.

E-commerce operations involve key data such as inventory, price, and order amount.

The system must ensure that:

- Inventory accuracy
- Order amount accurate
- Product status accurate
- Payment status accurate

To avoid problems such as overselling, underselling, or abnormal order status.

3. Prioritize ensuring back-end operational capabilities.

Products cannot be managed, inventory cannot be maintained, and even with a fully functional front-end, sales cannot proceed normally.

Therefore, backend product management, inventory management, and order management capabilities are essential features of an MVP.

4. Control development complexity

The MVP stage does not pursue feature richness, but rather the rapid validation of the business model.

Therefore, the following capabilities will not be included in the first phase of construction:

- Membership System
- Coupon system
- Marketing campaign system
- Product review system
- Personalized recommendation system
- Live streaming e-commerce capabilities
- Social sharing ability

The above features will be gradually planned in subsequent versions based on business development.

Product success criteria

After the MVP is launched, the product should be able to meet the following objectives:

User-side goals

1. Users can independently complete the product browsing and purchasing process.
2. Users can check order status
3. Users can view logistics information

Operational goals

1. Operations staff are able to manage products independently
2. Operations personnel are able to maintain inventory
3. Operations staff are able to process orders and ship goods.

Business side goals

1. Complete transaction closed-loop verification
2. Supporting the initial sales of goods on the platform
3. Collect user behavior data and business data
4. Provide a basis for subsequent product iterations

Through the above objectives, this project will complete the construction of the core capabilities of a basic e-commerce platform, from product display to order fulfillment, laying the foundation for subsequent business growth and functional expansion.

1.2 User Role Definition

To ensure the system design covers core business scenarios, it is necessary to clearly define the main user roles and their usage objectives. Based on the characteristics of e-commerce platform business, this system primarily involves three types of users:

1. Consumer (Customer / Buyer)
2. Operations Staff / Merchant Staff
3. Platform Admin

Different roles have different objectives, therefore different functional support is required.

1.2.1 Consumer (Customer / Buyer)

User Description

Consumers are the core users of e-commerce platforms and the direct source of their revenue.

Consumers browse products, compare product information, complete order placement and payment through the platform, and track order status and logistics information after purchase.

For consumers, the core value of the platform lies in enabling them to complete the shopping process conveniently, quickly, and securely.

User goals

Consumers hope:

1. Quickly find the products you need to buy
2. Get accurate and complete product information
3. Complete the purchase at a reasonable price
4. Stay informed about order status
5. Received the goods smoothly
6. Get a refund or after-sales support if problems occur.

User pain points

Pain points	illustrate
Product information incomplete	Unable to determine whether the product meets the requirements
Low search efficiency	Difficulty in quickly finding the target product
Inaccurate inventory information	Out of stock found when placing order
The payment process is complex.	This caused users to abandon their purchase.
Order status is not transparent	Users cannot track logistics progress.
The after-sales process is complex.	Decrease user satisfaction

Core use cases

Scenario 1: Browsing Products

After entering the platform, users can browse the product list and learn about the products currently on sale.

Features involved: Product list page, category filtering, product search

Scenario 2: Viewing product details

Users can view product information on the product details page.

Functions involved: Product image display, product name, product price, SKU specifications, inventory information, sales volume display

Scenario 3: Add to cart

Users add multiple items to their shopping cart and check out together.

Functions involved: Add to cart, modify quantity, delete item

Scenario 4: Buy Now

Users purchase individual items directly.

Functions involved: SKU selection, address confirmation, order submission

Scenario 5: Payment Order

Users complete their purchases via online payment.

Functions involved: Payment page, payment status feedback

Scenario 6: Order Tracking

Users can check the order processing progress.

Functions involved: Order list, order details, logistics tracking

Scenario 7: Refund Request

Users can apply for a refund if they meet the conditions.

Functions involved: Initiating a refund and viewing the refund progress.

User Value Summary

For consumers, the system needs to provide a one-stop shopping experience from product discovery to order completion, and ensure that:

- Product information transparency
- Accurate inventory information
- Smooth transaction process

- Order status is traceable

1.2.2 Operations Staff

User Description

Operations staff are responsible for the daily operations during the product sales process and play a crucial role in ensuring the normal operation of the platform.

Operations staff manage products, inventory, orders, and shipping processes through the back-end system to ensure that consumers can complete their shopping smoothly.

User goals

The operations team hopes that:

1. Quickly create products
2. Flexible maintenance of product information
3. Real-time inventory management
4. Efficient order processing
5. Arrange delivery in a timely manner
6. Quickly process abnormal orders

User pain points

Pain points	illustrate
Low product maintenance efficiency	Product update time
Chaotic inventory management	This could lead to overselling or stockouts.
Increased order volume makes processing difficult	Impact on contract performance efficiency
The shipping process is complex.	Increase operating costs
Lack of data support	Difficulty in analyzing business situation

Core use cases

Scenario 1: Creating Products

Operations staff create new products and enter product information.

Functions involved: Product creation, image upload, product category settings, SKU settings

Scenario 2: Editing Products

Operations staff update product prices, inventory, or status.

Functions involved: Product editing, product listing and delisting.

Scenario 3: Inventory Management

Operations staff maintain product inventory.

Functions involved: Inventory adjustment, inventory query, inventory alerts

Scenario 4: Order Management

Operations staff view and process orders.

Functions involved: order inquiry, order status management, and handling of abnormal orders.

Scenario 5: Arranging shipment

The operations staff completed the shipping process.

Functions involved: Creating a tracking order, entering the tracking number, and updating the shipping status.

Scenario 6: Processing Refunds

Operations staff review refund requests.

Functions involved: Refund approval, refund status update

User Value Summary

For operations personnel, the system needs to provide an efficient and stable operational backend to help them complete product sales and order fulfillment, improve operational efficiency, and reduce labor costs.

1.2.3 Platform Admin

User Description

The platform administrator is responsible for the management and maintenance of the entire system, mainly focusing on the platform's operation, access control, and handling of abnormal issues. During the MVP phase, platform administrators do not directly participate in the transaction process, but they are required to ensure the stable operation of the system.

User goals

The platform administrator hopes that:

1. Management backend account permissions
2. View the overall operation of the platform
3. Monitoring key business metrics
4. Handling abnormal orders and system issues

User pain points

Pain points

Disorganized access control

Lack of overall data view

It is difficult to investigate abnormal orders.

Delay in system problem detection

illustrate

There are security risks.

Difficulty in grasping the operational situation

Impact on user experience

Increased operational risks

Core use cases

Scenario 1: User and Permission Management

Manage backend accounts and role permissions.

Functions involved: Account management, role management, permission configuration

Scenario 2: Data Monitoring

View the platform's operational data.

Functions involved: Data dashboard, core indicator statistics

Scenario 3: Handling Abnormal Orders

Handling abnormal situations during the transaction process.

Functions involved: Abnormal order query, status repair, manual intervention

User Value Summary

For platform administrators, the system needs to provide comprehensive management and data monitoring capabilities to ensure the platform operates securely, stably, and efficiently.

1.3 Summary of User Role Relationships

User Role	Core Objectives	Main functional modules
consumer	Complete purchase	Products, shopping cart, orders, payment
Operations personnel	Managing products and orders	Product management, inventory management, order management, shipping management
Platform Administrator	Management platform operation	User management, access control, and data monitoring

These three types of roles together constitute a complete e-commerce business loop:

Consumers generate order demand;

Operations staff complete product sales and order fulfillment;

Platform administrators ensure the platform's stable operation.

By clearly defining the responsibilities and needs of each role, we can provide a basis for subsequent MVP feature design and priority allocation.

II. MVP Feature List and Priority Design

2.1 Priority Definition

In the MVP stage, the core goal of product design is not to implement all the capabilities of a complete e-commerce platform at once, but to prioritize ensuring that the core transaction chain can operate normally within limited time and resources.

The core transaction process of this system is as follows:

Browse products → Add to cart / Buy now → Submit order → Pay → Ship → Order complete

Therefore, functional priorities will be allocated around this transaction loop.

Priority	Definition	Judgment criteria
P0	Must-be-launched features	Without this feature, users cannot complete the core transaction process, or operations staff cannot support product sales and order fulfillment.
P1	Suggested features to be launched	It does not affect the basic transaction loop, but can significantly improve user experience, operational efficiency, or anomaly handling capabilities.
P2	Subsequent iteration features	It's not essential for closing the transaction loop; it's more about enhancing growth, marketing, retention, or refined operational capabilities.

P0Functional judgment logic

The P0 function primarily serves the purpose of determining whether an order can be completed.

For example:

- Without a product list, users cannot find products;
- Without a product details page, users cannot decide whether to purchase.
- Without the order submission function, a transaction cannot be generated;
- Without payment functionality, the platform cannot complete the collection of payments;
- Without backend product management, the platform has no products to sell.
- Without inventory management, the system is prone to overselling or stockouts.
- Without order management and shipping management, orders cannot be fulfilled.

Therefore, the P0 function must be developed as a priority.

P1Functional judgment logic

The P1 function primarily serves to "ensure that transactions can be completed more smoothly and efficiently".

For example:

- Product search and category filtering can improve the efficiency of users finding products;
- The order cancellation and refund functions can handle abnormal transaction scenarios;

- Data dashboards can help operations staff understand the business situation;
- User management can support basic platform governance.

While these functions are not essential for the core transaction loop, they can impact user experience, operational efficiency, and platform stability, making them suitable for placement in P1.

P2Functional judgment logic

The P2 function primarily serves to "improve growth and long-term operational capabilities".

For example:

- Coupons can boost conversions, but they are not a basic requirement for transactions.
- Product reviews can increase user trust, but the MVP stage can be postponed.
- Features such as membership systems, referral systems, and marketing campaigns can increase repeat purchases and user stickiness, but they are complex to develop and not suitable for investing too many resources in the first phase.

Therefore, the P2 function is best planned after the core transaction chain is stable.

2.2 User-side function list and priority

The user-facing features are primarily aimed at consumers, with the goal of helping users complete the entire shopping process from browsing products to placing an order.

Module	Function	Priority	reason
Account System	Login / Register	P0	Users need to be identified through their accounts, and the system needs to link shopping carts, orders, payment records, and after-sales records. Without an account system, user orders are difficult to manage.
Product Center	Product List	P0	The product list is the primary entry point for users to discover products. Without a product list, users cannot browse products on the platform.
Product Center	Product details page	P0	Users need to view information such as product name, price, image, SKU, and inventory before making a purchase decision.

Product Center	Product status display	P0	Users need to know whether a product is available or sold out. Purchases of delisted or out-of-stock items should be restricted to avoid invalid orders.
SKU selection	SKU selection	P0	For multi-specification products, users need to select specific SKUs, such as color and size. Orders must be linked to specific SKUs for inventory deduction and fulfillment.
shopping cart	add to the cart	P0	The shopping cart allows users to temporarily store items and supports the unified checkout of multiple items, making it an important function in the basic transaction process.
shopping cart	Modify purchase quantity	P0	Users need to adjust the purchase quantity before placing an order, and the system also needs to check the inventory based on the purchase quantity.
shopping cart	Delete product	P1	Users can delete items they don't want to buy, improving the shopping cart management experience without affecting the basic purchase process.
Order System	Submit an order	P0	Submitting an order is a crucial step in transforming a shopping behavior into a transaction, and the system needs to generate the order at this stage.
Order System	Order Amount Calculation	P0	The system needs to calculate the order amount based on information such as product price, quantity purchased, and shipping costs to ensure accurate payment.
Address System	Shipping address entry / selection	P0	For physical goods to be shipped, a delivery address is required; therefore, address information is a necessary condition for order fulfillment.
payment system	Online payment	P0	Payment is a crucial step in completing the transaction loop. Without payment functionality, an order cannot be converted into a valid transaction.
payment system	Payment result display	P0	Users need to know for sure whether the payment was successful, and the system also needs to update the order status based on the payment result.

Order System	Order List	P0	Users need to view their historical orders and current order status.
Order System	Order Details	P0	Users need to view complete information such as order items, amount, address, payment status, and logistics status.
Logistics system	Logistics information viewing	P1	Users can view shipping status and logistics progress, improving order transparency. The MVP can initially display the basic tracking number and shipping status.
Search system	Product Search	P1	Search can help users quickly find their target products and improve shopping efficiency, but it is not absolutely necessary in the MVP stage when there are few products.
Search system	Category Filtering	P1	Categorization and filtering can improve product discovery efficiency, but these can be optimized after the basic functions of the product list are completed.
After-sales system	Cancel order	P1	Users may need to cancel orders before payment or delivery, which helps in handling abnormal transaction scenarios.
After-sales system	Refund Request	P1	Refunds are an important capability in after-sales scenarios, but in the MVP stage, a simple refund process can be supported first, without the need for complex after-sales rules.
Marketing System	coupons	P2	Coupons can help improve conversion rates, but they do not affect the core transaction process and can be implemented in subsequent marketing versions.
User Interaction	Product Reviews	P2	Product reviews can enhance user trust, but in the MVP stage, the focus should be on the transaction loop first, and review capabilities can be added later.
Recommendation system	Personalized recommendations	P2	Recommendation systems rely heavily on data and algorithms, resulting in high development costs, making them unsuitable for the first version of an MVP.

2.3 List and Priority of Backend Operation Functions

The back-end operation functions are mainly aimed at operations personnel and merchant employees, with the goal of supporting product listing, inventory maintenance, order processing, and delivery fulfillment.

Module	Function	Priority	reason
Product Management	Product creation	P0	Products must be created before they can be displayed and sold on the front end; this is a fundamental function of e-commerce systems.
Product Management	Product Editing	P0	Operations staff need to modify product names, prices, images, descriptions, and other information to ensure the accuracy of product information.
Product Management	Product delisting and relisting	P0	Product status affects whether a user can purchase it. Products that have been delisted cannot continue to be sold to avoid generating invalid orders.
Product Management	Product Image Management	P0	Product images directly influence users' purchasing decisions and are an important part of the product details page.
SKU Management	SKU creation	P0	For products with multiple specifications, the system needs to distinguish different colors, sizes or models by SKU and manage prices and inventory separately.
SKU Management	SKU Price Maintenance	P0	Different SKUs may have different prices, and the system needs to support SKU-level price maintenance.
Inventory Management	Inventory maintenance	P0	Inventory is crucial data for order verification and fulfillment. Inaccurate inventory can lead to overselling or failed order placements.

Inventory Management	Inventory deduction	P0	After a user places an order or makes a payment, the system needs to deduct inventory according to business rules to ensure the accuracy of inventory data.
Inventory Management	Inventory warning	P1	Reminding operations to replenish stock when inventory falls below a certain threshold can improve operational efficiency, but it is not a necessary function for the core transaction loop of an MVP.
Order Management	Order List	P0	Operations staff need to view all orders and process them according to their status.
Order Management	Order Details	P0	Operations staff need to view information such as order items, user information, shipping address, and payment status in order to fulfill the order.
Order Management	Order Status Management	P0	The system needs to support the order status transition from pending payment, paid, shipped to completed.
Shipment Management	Create shipping information	P0	Orders that have been paid for need to enter the shipping process, and operations staff must be able to fill in the logistics information and ship the goods.
Shipment Management	Logistics tracking number entry	P0	The tracking number is used by users to check the logistics status and is also an important piece of information for order fulfillment.
Shipment Management	Shipping status update	P0	After shipment, the order status needs to be updated from "paid" to "shipped" to ensure that users and operations staff see the same status.
After-sales management	Refund review	P1	Refund processing falls under after-sales service capabilities and is important for user experience, but it can be simplified during the MVP stage.

User Management	User account query	P1	The backend can query user information, which is convenient for customer service or operations to handle order issues, but it is not essential for the basic transaction loop.
Access Management	Backend role permissions	P1	Different back-end personnel should have different operating permissions, such as product management, order management, and administrator permissions, which helps reduce the risk of accidental operation.
Data dashboard	Order data statistics	P1	Data dashboards help operations staff understand metrics such as order volume, payment conversion, and GMV, but basic statistics can be done in the first phase.
Data dashboard	Product sales data	P1	It helps operations determine which products sell well, providing a basis for subsequent product operations.
Marketing Management	Coupon Management	P2	Coupons are a marketing capability and do not affect the basic transaction loop; they can be iterated upon later.
Content Management	Homepage Banner Management	P2	Homepage placements can improve conversion rates, but they are not a necessary feature in the first stage.
Refined Operations	User Tiered Management	P2	User segmentation is suitable for platforms that have reached a certain user base; it is not a priority during the MVP stage.

2.4 Summary of MVP Core Functionality Scope

Based on the above priority analysis, the first version of the MVP should prioritize covering the following core functionalities:

User-side core functions

Function	Should it be included in MVP?	illustrate
Login / Register	yes	Supports user identification and order association
Product List	yes	Supports users browsing products
Product details page	yes	Users can view product information and make purchase decisions.
SKU selection	yes	Supports purchasing products in multiple specifications
add to the cart	yes	Supports users to temporarily store goods and settle payments in one go.
Submit an order	yes	Support users in generating orders
Online payment	yes	Support transaction completion
Order List / Order Details	yes	Users can view order status.
Logistics information viewing	Simplified inclusion	In the MVP stage, the shipping status and tracking number can be displayed first.
Cancel order / Refund	Simplified inclusion	In the MVP phase, priority will be given to supporting cancellations of unpaid orders and basic refund requests.
Coupons / Product Reviews / Recommendations	no	This is a feature for future growth and user experience optimization.

Backend core functions

Function	Should it be included in MVP?	illustrate
Product creation and editing	yes	Support product listing and sales
Product delisting and relisting	yes	Controlling whether goods are available for purchase
SKU Management	yes	Supports multiple product specifications
Inventory Management	yes	Supports order verification and inventory deduction
Order Management	yes	Support order processing
Shipment Management	yes	Support order fulfillment
User Management	Simplified inclusion	Supports basic account query
Data dashboard	Simplified inclusion	In the MVP stage, core order and sales data can be displayed first.
Marketing Management	no	Subsequent version development

2.5 Description of features not included in this MVP

To control the scope of development and ensure the core transaction chain goes live as soon as possible, the following features will not be included in the first version of the MVP.

Function	Reasons for not including	Subsequent plans
Coupon system	It does not affect the basic purchase process, but it involves discount rules, order amount calculation, and marketing configuration, making it quite complex.	It can be built as a marketing module after the transaction chain is stable.
Membership System	It does not affect users' ability to complete basic purchases, and requires additional rules such as points, levels, and benefits.	It can be built after the user base grows.
Product Reviews	This helps build trust, but in the MVP stage, you can first support purchase decisions with product details.	A review section can be added after the order is completed.
Personalized recommendations	It requires user behavior data and recommendation algorithm support; initial data is insufficient.	After accumulating browsing and purchase data on the platform, it can be used to build...
Marketing campaign	The development costs are high, involving activity rules, activity inventory, activity pricing, and the activity page.	This can be used as a basis for planning subsequent growth versions.
Live streaming e-commerce	It does not fall under basic e-commerce transaction capabilities and requires a high level of content operation and technical skills.	Not considering at the moment
Social sharing	It can enhance reach, but it does not affect users' ability to complete a purchase.	Further increases will be made based on growth demand.

2.6 Summary of Functional Priority Design

The first version of the MVP was designed to ensure that the e-commerce platform could complete the most basic and important transaction loop.

From the user's perspective, the system needs to support consumers in completing the following:

Browse products → View details → Select SKU → Add to cart / Buy now → Submit order → Pay → View order

From an operations perspective, the system needs to support operations personnel in completing the following tasks:

Create product → Maintain SKU and inventory → Manage orders → Arrange shipment → Update order status

Therefore, the P0 function mainly revolves around products, inventory, orders, payments, and shipping; the P1 function is mainly used to improve user experience and operational efficiency; and the P2 function serves more for subsequent growth and refined operations.

This prioritization ensures that the product can be launched quickly in the first phase with low complexity, while reserving room for subsequent feature expansion.

III. Core Process Design

This section selects two core processes for detailed design:

1. User order placement and payment process
2. Product creation and publishing process

The reason for choosing these two processes is that the user order placement and payment process is the core transaction link of the e-commerce system, which directly affects order conversion and revenue; the product creation and publishing process is the prerequisite for the transaction link. If product information, SKU, inventory and status management are incomplete, the front-end transaction process cannot operate normally.

3.1 User order placement and payment process

3.1.1 Process Objectives

The goal of the user order and payment process is to support consumers from selecting products to completing payment and generating a valid order.

This process needs to ensure that:

- The product is available for purchase.
- SKU selection is effective
- Sufficient inventory
- Order amount calculation accurate
- Payment status can be synchronized correctly.
- Order status can be transferred according to business rules.

This process is the core transaction link of the e-commerce system, which directly determines whether a user can complete the purchase.

3.1.2 Roles Involved

Role	illustrate
consumer	Select your products, submit your order, and complete payment.
system	Verify product status, inventory, price, address, and payment result.
payment platform	Process user payment requests and return payment results.
Operations personnel	Follow up on paid orders and arrange shipment.

3.1.3 Prerequisites

Before a user can proceed with the order placement process, the following prerequisites must be met:

1. The user has logged into the system.
2. The product is in the "on sale" state, meaning its status is ON_SALE.

3. The user has selected a valid SKU.
4. The number of purchases by a user is greater than 0.
5. The SKU inventory quantity is greater than or equal to the quantity purchased by the user.
6. The user has filled in or selected a valid shipping address.
7. Product prices and inventory information can be read normally by the system.

If the above conditions are not met, the system should prevent the user from submitting an order and display a clear error message to the user.

3.1.4 Normal Procedure

Scenario: A user clicks "Buy Now" on the product details page and completes the payment.

step	User actions / System actions	illustrate
1	User enters product details page	View product images, name, price, SKU, inventory, and other information.
2	User selects SKU	For example, choosing colors, sizes, and models.
3	User selects purchase quantity	The system determines whether a purchase is possible based on SKU inventory.
4	The user clicked "Buy Now"	Entering the order confirmation page
5	System checks product status	Determine if the product is still listed.
6	The system verifies SKU status and inventory.	Determine if the SKU is valid and if the inventory is sufficient.
7	User confirms shipping address	The address will be used for subsequent shipping.
8	The system calculates the order amount.	This includes the product price, shipping costs, and discount amounts. Coupons can be omitted during this MVP phase.
9	The user clicks "Submit Order"	System creates order

10	The system generates an order number.	The order status has changed to "Pending Payment".
11	The user enters the payment page	Select payment method and initiate payment.
12	Payment platform processes payment requests	Returns the payment success or failure result.
13	Payment successful	The system has updated the order status to "Paid".
14	Users view payment results	The page displays "Payment successful".
15	Order enters the pending shipment process	The operations staff will arrange subsequent shipments.

3.1.5 State Changes

Order status change

Business Node	Order status	illustrate
User successfully submitted order.	PENDING_PAYMENT (Pending Payment)	The order has been generated, but the user has not yet completed payment.
User payment successful	PAID (Paid)	The payment platform has confirmed successful payment, and the order has entered the pending shipment stage.
The user failed to pay within the specified time.	CANCELLED	The system automatically canceled the order.
User cancels unpaid order	CANCELLED	User cancels order
Request a refund after payment	REFUNDING	User initiates refund request

Refund completed

REFUNDED

Refund processed

3.1.6 Inventory Change Rules

Inventory management is a crucial part of the order placement process. In the MVP phase, it's recommended to use a method of "locking inventory upon order submission and releasing it after payment timeout."

Business Node	Inventory handling methods	illustrate
Users browse products	No deduction of inventory	Only currently available inventory is displayed.
Add to cart by user	No deduction of inventory	The shopping cart only temporarily stores items and does not guarantee a successful purchase.
User submits order	Lock in inventory	To prevent overselling caused by other users purchasing at the same time.
User payment successful	Officially deduct inventory	The order becomes a valid completed order.
User payment timed out	Release locked inventory	Avoid inventory being tied up for a long time
User cancels unpaid order	Release locked inventory	Order invalid, inventory restored
Refund after payment	Whether to replenish inventory depends on business rules.	If the goods have not been shipped, they can usually be replenished from inventory; if they have already been shipped, the process will depend on the outcome of the return.

Note: If the system's initial complexity is limited, a "deduct inventory upon successful payment" approach can be used. However, this method is more prone to overselling in high-concurrency scenarios. Therefore, from a product design perspective, locking inventory upon order submission is more prudent.

3.1.7 Abnormal Scenario Design

Abnormal Scenario 1: Insufficient Inventory

Triggering conditions

When a user submits an order, the system finds that the selected SKU is out of stock.

Possible reasons include:

- The product is still in stock when the user sees it on the product details page, but it has already been purchased by another user before the user submits the order;
- The number of users purchased exceeded the current SKU inventory;
- The inventory of items in the shopping cart has changed;
- The inventory displayed on the front end is not synchronized with the actual inventory on the back end.

System processing

1. The system is preventing order creation.
2. The payment process will not proceed.
3. The page indicates that the user's inventory is insufficient.
4. Users can return to the product details page or shopping cart to reselect the quantity.

Page prompt text

The current stock of this product is insufficient. Please adjust the quantity you wish to purchase or select another specification.

State change

The order was not created, therefore no order status change occurs.

Abnormal Scenario 2: The product has been removed from the store, but it still exists in the shopping cart.

Triggering conditions

The user added the item to their shopping cart, but the operations staff removed the item before the user could checkout.

System processing

1. When a user adds an item to their shopping cart, the system re-verifies the item's status.

2. If the product status is **OFF_SALE** If the item is not listed, it will be marked as "out of stock" in the shopping cart.
3. This item cannot be selected for checkout.
4. Users can delete the product or wait for it to be relisted.

Page prompt text

This product has been removed from the shelves and is temporarily unavailable for purchase.

State change

The order was not created, therefore no order status change occurs.

Business Description

The product information in the shopping cart cannot be used as the final basis for the transaction. The system must re-verify the product status, SKU status, price, and inventory before submitting the order, and the backend verification results should be used as the final judgment.

Abnormal Scenario 3: Payment Failure

Triggering conditions

After a user initiates a payment, the payment platform returns a payment failure result.

Possible reasons include:

- Insufficient user account balance;
- Bank card payment failed;
- The third-party payment platform is malfunctioning;
- The user canceled the payment.
- A network error prevented the payment from being completed.

System processing

1. The order remains in the "pending payment" status.
2. The system displays a payment failure message.
3. Users can re-initiate the payment.
4. If payment is not received by the deadline, the system will automatically cancel the order.

Page prompt text

Payment failed. Please try paying again or change your payment method.

State change

PENDING_PAYMENT(Pending payment) → Still pending PENDING_PAYMENT(Pending payment)

If subsequent payment times out:

PENDING_PAYMENT(Pending payment) → CANCELLED(Cancelled)

Abnormal Scenario 4: Payment Timeout**Triggering conditions**

The user submitted an order but failed to complete the payment within the specified time.

For example: Payment is not completed within 30 minutes of the order being created.

System processing

1. The system's scheduled task checks for unpaid orders that have timed out.
2. Update the order status to CANCELLED.
3. Release orders to lock in inventory.
4. When the user revisits the order details page, the page displays that the order has been cancelled.
5. Users who still wish to purchase need to place a new order.

Page prompt text

Your order payment has timed out and has been automatically cancelled. Please place a new order.

State change

PENDING_PAYMENT(Pending payment) → CANCELLED(Cancelled)

Inventory changes

Release the inventory locked for this order.

Abnormal Scenario 5: Payment Result Callback Delay

Triggering conditions

The user has completed the payment on the payment platform, but the system has not yet received a payment success callback.

System processing

1. The front-end page displays "Payment processing".
2. The system proactively queries the payment platform for payment results.
3. If payment is confirmed successful, the order status will be updated to...**PAID**.
4. If payment failure is confirmed, the order remains **PENDING_PAYMENT**.
5. If the payment is not successfully confirmed after the payment deadline, the decision to cancel the order will be made based on the final status of the payment platform.

Page prompt text

Payment confirmation in progress. Please refresh the page later to check the order status.

State change

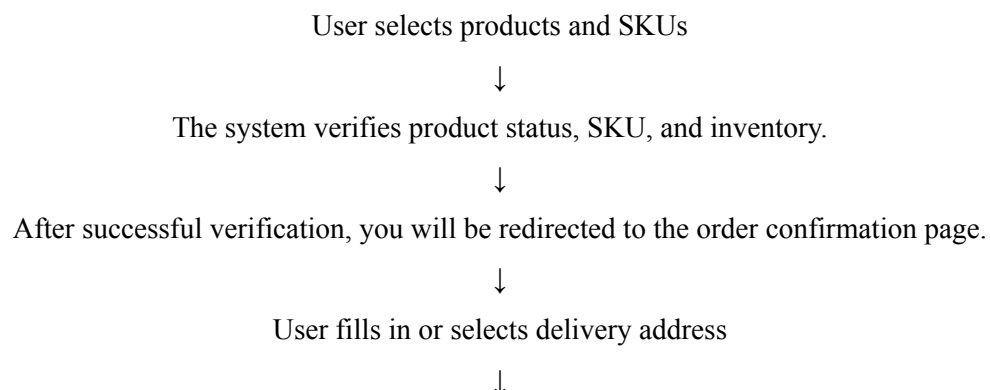
PENDING_PAYMENT(Pending payment) → **PAID**(Paid)

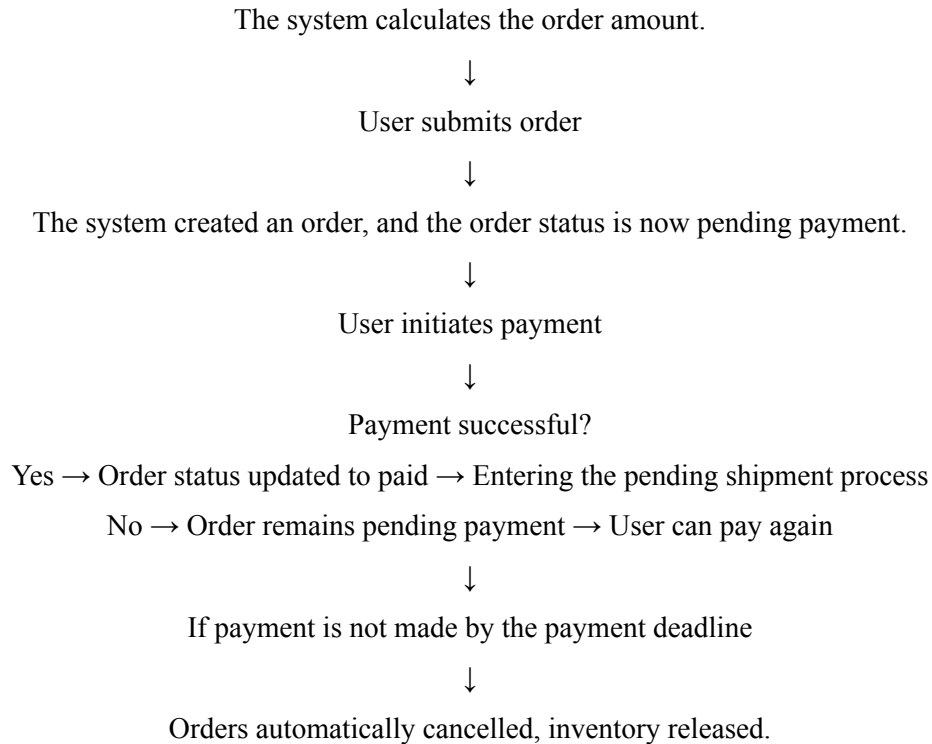
or:

PENDING_PAYMENT(Pending payment) → **CANCELLED**(Cancelled)

3.1.8 Flowchart Description

The user order placement and payment process can be described as follows:





3.1.9 Key Points of Product Design

In designing this process, product managers need to focus on the following issues:

1. Backend validation must be used as the final basis for judgment.

The product status and inventory displayed on the front end may be cached or delayed, so the back end must re-validate them when a user submits an order.

- Is the product listed?
- Is the SKU effective?
- Is the inventory sufficient?
- Has the price changed?
- Is the user address valid?

2. Order status must be clear.

Order status should accurately reflect the current business stage to avoid problems such as users having paid but the order still showing as pending payment, or merchants having shipped goods but users not being able to see the logistics information.

3. Payment anomalies must be recoverable.

Payment failures or delays in payment processing should not result in users losing their orders. Instead, users should be allowed to either re-pay or wait for the payment result to be synchronized.

4. Inventory management needs to prevent overselling.

Insufficient inventory is one of the most common problems in e-commerce systems. The system needs to verify inventory when an order is submitted and lock or deduct inventory according to business rules.

3.2 Product Creation and Shipping Process

3.2.1 Process Objectives

The goal of the product creation and publishing process is to support operations staff in creating products in the backend and publishing them to the frontend for consumers to browse and purchase.

This process needs to ensure that:

- Complete basic product information
- Product price valid
- SKU information is correct
- Accurate inventory quantity
- Product status is controllable
- The listed products can be displayed normally on the front end.

Product creation and publishing are the initial steps in the e-commerce transaction process. Only when product information is accurate, inventory is available for sale, and the product status is normal can the user's purchase process have a foundation.

3.2.2 Roles Involved

Role	illustrate
Operations personnel	Create products, maintain product information, set SKUs and inventory, and perform the put-away operation.
system	Validate product fields, save product information, control product status, and synchronize front-end display.
consumer	Browse and purchase listed products on the front end.

Platform Administrator	Manage product permissions or handle abnormal products when necessary.
------------------------	--

3.2.3 Prerequisites

Before creating a product, operators need to meet the following conditions:

1. The operations staff has logged into the backend system.
2. Operations personnel have product management privileges.
3. The product category already exists.
4. Basic information such as product images, product names, prices, and inventory has been prepared.
5. If a product has multiple specifications, the SKU rules need to be clearly defined in advance, such as color, size, model, etc.

3.2.4 Definition of Product Status

During the creation and publication of products, it is necessary to control whether they can be seen and purchased by users through status control.

Product status	meaning	Front-end performance
DRAFT	Product information is incomplete and is only visible in the backend.	Invisible to the front end
PENDING_REVIEW (Pending review)	The product has been submitted and is awaiting review.	Invisible to the front end
ON_SALE (Currently Available)	The product has been listed and is available for sale.	Visible and available for purchase on the front end
OFF_SALE (No longer available)	Products were proactively removed from the platform by the operations team or removed from the system.	Purchases are not available at the front desk.
SOLD_OUT (Sold Out)	Goods inventory is 0	The items can be displayed on the front desk but cannot be purchased.

Note: If no review mechanism is designed in the MVP stage, the PENDING_REVIEW state can be omitted. After the operations staff creates the product, they can directly save it as a draft or put it on the shelves.

3.2.5 Normal Procedure

Scenario: Operations staff create and publish products.

step	Operation / System Actions	illustrate
1	Operations personnel access the backend product management page	View product list
2	Click "Create New Product"	Enter the product creation page
3	Fill in the basic product information	This includes product name, product category, product images, and product description.
4	Set product price	Fill in the selling price and original price
5	Set SKU information	For example, color, size, model, etc.
6	Set SKU prices and inventory.	Different SKUs can have different prices and inventory levels.
7	System verifies product information	Validate the completeness of required fields, price, inventory, and SKU.
8	Operations personnel store goods	The product is currently in draft status.
9	The operations staff clicked "List".	The system re-verifies the product's eligibility for sale.
10	The system updates the product status to ON_SALE.	The product is now available for sale.
11	The product is displayed in the front-end product list and product details page.	Users can browse and purchase

3.2.6 State Changes

Product status changes

Business Node	Product status	illustrate
Created a product but not fully filled in	DRAFT	Products are only visible in the backend.
Fill in the product information completely and save.	DRAFT	The product is not yet available.
The product passes verification and is clicked to be listed.	ON_SALE (Currently Available)	The product is visible and available for purchase on the front end.
The operators took the initiative to remove it.	OFF_SALE (No longer available)	The product cannot be purchased at the front desk.
Goods inventory is 0	SOLD_OUT (Sold Out)	The product is displayed on the front end, but the "Buy" button is unavailable.
Operations to replenish inventory and restock	ON_SALE (Currently Available)	The product is now available for purchase.

3.2.7 Product Field Design

When creating a product, operations staff need to fill in the following fields:

Field	Is this field required?	illustrate
Product Name	yes	Used for front-end display and search
Product Categories	yes	Used for product classification and filtering
Product Images	yes	Used for displaying product lists and detail pages
Product Description	yes	Help users understand product features
Selling price	yes	Actual purchase price for users
Original price	no	Used for price comparison display
SKU Name	yes	For example, white, black, size M, size L, etc.

SKU Price	yes	Different SKUs may have different prices
SKU inventory	yes	Used for order inventory verification
Product status	yes	Controlling whether goods are available for sale
Shipping rules	MVP optional	Used to calculate the total order amount
After-sales rules	MVP optional	For refunds and after-sales instructions

3.2.8 Business Rules

Rule 1: Product name cannot be empty.

The product name is the core information that users need to identify a product. If the product name is empty, the system will not allow it to be saved or listed.

Error message: Product name cannot be empty.

Rule 2: Each product must have at least one image uploaded.

Product images influence users' purchasing decisions and are also the basic information displayed in product listings and detail pages.

Error message: Please upload at least one product image.

Rule 3: The selling price must be greater than 0.

The selling price must be a valid amount and cannot be 0 or negative.

Error message: The selling price of the product must be greater than 0.

Rule 4: SKU inventory cannot be less than 0

Inventory quantity cannot be negative. If inventory is 0, the product or SKU cannot be purchased.

Error message: SKU inventory cannot be less than 0.

Rule 5: Each listed product must have at least one valid SKU.

If there are no available SKUs for the product, the order cannot be completed.

Error message: Please set at least one valid SKU before listing.

Rule Six: Products must pass an integrity check before being listed.

The system needs to validate key fields such as product name, image, price, SKU, and inventory. If the information is incomplete, the product can only be saved as a draft and cannot be listed.

Error message: Product information is incomplete and cannot be listed at this time.

Rule 7: Products that have been removed from the shelves cannot be purchased.

When a product is in the OFF_SALE state, the front end should restrict its purchase.

Front-end display: This product is no longer available and cannot be purchased at this time.

Rule 8: You cannot purchase when inventory is 0.

When the total inventory of the product or the inventory of the SKU selected by the user is 0, the purchase button is grayed out, and the user is not allowed to submit an order.

Front-end display: Current product inventory is insufficient.

3.2.9 Abnormal Scenario Design

Abnormal Scenario 1: Product information is incomplete

Triggering conditions

When the operations staff clicked "List," the system found that the product was missing a required field.

Possible missing fields include:

- Product Name
- Product Images
- commodity prices
- SKU Information
- SKU inventory
- Product Categories

System processing

1. The system prevents the product from being listed.
2. The product status remains DRAFT.
3. The page annotation is missing a field.

4. The operator is prompted to provide complete information.

Prompt text

The product information is incomplete. Please complete it before listing.

State change

DRAFT(Draft) → Still in use**DRAFT**(draft)

Abnormal Scenario 2: SKU inventory is 0**Triggering conditions**

When operators set up product SKUs, the inventory of a certain SKU is 0.

System processing

1. Goods may be stored.
2. If all SKUs have zero inventory, the product cannot be listed, or it will be displayed as sold out if listed.
3. If some SKUs have zero inventory, then those SKUs cannot be purchased; other SKUs with inventory can still be purchased.

Prompt text

This SKU has zero stock, so users cannot purchase this specification at this time.

State change

If all SKUs have zero inventory:

DRAFT(Draft) → **SOLD_OUT**(Sold out) or hold**DRAFT**(draft)

If some SKUs have zero inventory:

Product status can be **ON_SALE** However, the corresponding SKU cannot be purchased.

Abnormal Scenario 3: Abnormal Product Prices

Triggering conditions

The product price entered by the operator does not comply with the rules.

For example:

- The selling price is 0
- The selling price is negative.
- SKU price is empty
- The original price was lower than the selling price and it was not explicitly permitted.

System processing

1. The system does not allow uploading.
2. The page displays an error message indicating that the price field is incorrect.
3. The operations staff modified the price and resubmitted it.

Prompt text

The product price setting is incorrect. Please check the selling price and SKU price.

State change

DRAFT(Draft) → Still in useDRAFT(draft)

Abnormal Scenario 4: Products are removed from the platform after being listed.

Triggering conditions

The product has been listed, but the operations staff has taken it down due to reasons such as inventory, price, product quality, or business reasons.

System processing

1. Product status updated toOFF_SALE.
2. The product details page on the front end displays "Product is no longer available".
3. Items cannot be added to the shopping cart or purchased immediately.
4. The item that is already in the user's shopping cart is marked as "out of stock".
5. When a user submits an order, the system re-verifies the product status and prevents the purchase.

Front desk notification text

This product has been removed from the shelves and is temporarily unavailable for purchase.

State change

ON_SALE(Currently available) →OFF_SALE(No longer available)

Abnormal Scenario 5: The product sells out after being listed.

Triggering conditions

During the sales process, the total inventory of goods or the inventory of a certain SKU becomes 0.

System processing

1. The product details page shows an inventory of 0.
2. The corresponding SKU is either unselectable or the "Buy After Selection" button is grayed out.
3. If all SKUs have zero inventory, the product will be displayed as sold out.
4. Users are unable to submit orders.
5. Operations staff can replenish inventory in the backend.

Front desk notification text

The item is currently sold out.

State change

ON_SALE(Currently available) →SOLD_OUT(Sold out)

After replenishing inventory:

SOLD_OUT(Sold out) →ON_SALE(Currently available)

3.2.10 Flowchart Description

The product creation and publishing process can be described as follows:

Operations personnel access the backend product management



Click to create a new product



Fill in basic information such as product name, category, image, and description.



Set SKUs, prices, and inventory.



Click Save



The system performs field integrity verification.



Verification passed?

No → Save as draft, with prompts for additional information.

Yes → Product saved successfully



Operations staff click to put on the shelf



The system will re-verify whether the product meets the conditions for sale.



Verification passed?

No → The product remains in draft or unsaleable status.

Yes → Product status updated to "Listing Now"



The product is displayed in the front-end product list and product details page.



Users can browse and purchase

3.2.11 Key Points of Product Design

1. The condition of the goods must be clearly stated.

Product status directly affects front-end display and user purchasing behavior.

The system must clearly distinguish:

- Draft products: Visible only in the backend
- Listed products: Visible and available for purchase on the front end.
- Removed from sale: Not available for purchase
- Sold-out items: can be displayed but not purchased.

If the status definition is not clear, it can easily lead to users seeing products that cannot be purchased, or operations staff mistakenly publishing incomplete products to the front end.

2. SKU is the core unit for inventory and price management.

In e-commerce systems, what users ultimately purchase is usually not an abstract product, but a specific SKU.

For example: Product: Bluetooth Earphones

SKU 1: White, Price: 199, Stock: 10

SKU 2: Black, Price 209, Stock 15

Therefore, orders, inventory deductions, and price displays should all be based on SKUs as the core unit, rather than relying solely on total product inventory.

3. Integrity verification must be performed before listing.

Before listing products, the system must verify that the product information is complete; otherwise, the following problems may occur:

- The lack of product images affects users' purchasing decisions.
- The product has no price listed, so I cannot place an order.
- The item is out of stock and cannot be purchased.
- The product has no SKU, and the order cannot be bound to a specific specification.

Therefore, the verification rules for listing products should be more stringent than those for storing products.

4. The front-end display must be consistent with the back-end status.

After operations staff modify product status in the backend, the frontend should synchronize the changes promptly. For example:

- Once a product is removed from the shelves, purchasing will be immediately disabled on the front desk.
- When an SKU's inventory reaches 0, the front end should display "Sold Out".
- If the price of a product changes, the price must be re-verified before the order is submitted.

The front-end display can allow some caching, but the order submission process must be based on the real-time verification results from the back-end.

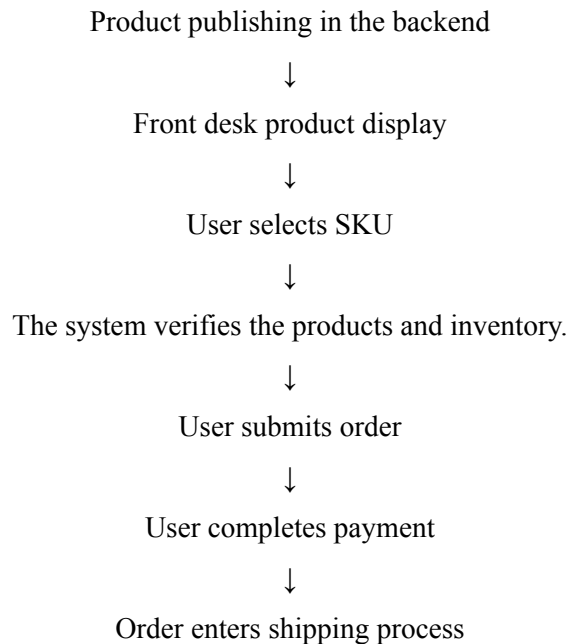
3.3 Relationship between the two processes

The user order and payment process depends on the product creation and publishing process.

A user can only make a purchase if the product in the backend meets the following conditions:

1. The product has been created;
2. Product status is **ON_SALE**;
3. The product must have at least one valid SKU;
4. SKU inventory is greater than 0;
5. The product price is valid;
6. Product information can be displayed normally on the front end.

The relationship between the two processes can be understood as follows:



Therefore, the product management process determines "whether there are products to sell," while the order placement and payment process determines "whether the user can complete the purchase." Together, they constitute the core business foundation of an e-commerce MVP.

IV. Simplify PRD Design: Product Details Page

4.1 Function Name

Product Detail Page

4.2 Functional Background

The product details page is a key page in an e-commerce system that connects "product browsing" and "order placement".

After seeing a product in the product list, users need to go to the product details page to learn more about the product, including product images, product name, price, SKU specifications, inventory, sales volume, etc. Only after fully understanding the product information will users decide whether to add it to their cart or make an immediate purchase.

Therefore, the product detail page not only serves the function of displaying products, but also guides purchase decisions, making it an important page that affects user conversion rates.

In this MVP version, the core goal of the product details page is not to achieve complex marketing displays, but to ensure that users can clearly understand the product information and smoothly enter the subsequent purchase process.

This feature needs to address the following key issues:

1. Can users clearly see what the product is?
2. Can users know the current price of a product?
3. Can the user select the correct SKU?
4. Can users know whether the current product or SKU is in stock?
5. Can users add items to their cart or purchase them immediately when the items are available for sale?
6. Can the system provide a clear prompt when a product is taken off the shelves, sold out, or in short supply?

The product details page is a user-side P0 function because if users cannot view the product details, they cannot make a purchase decision, and the subsequent shopping cart, order submission, and payment processes cannot proceed smoothly.

4.3 User Objectives

Target users: The primary target users of this feature are consumers (Customers/Buyers).

User core objectives

After entering the product details page, users hope to be able to quickly complete the following tasks:

1. Understand the basic information about the product;
2. View product images;
3. View the current selling price and original price;
4. Select product specifications, such as color, size, model, etc.;
5. Check the inventory of the selected SKU;
6. Determine whether a product is worth buying;
7. Add items to your shopping cart;
8. Proceed directly to the order placement process.

User usage scenarios

Scenario 1: A user navigates from the product list to the product details page.

Users browse products on the product list page, and after clicking on a product card, they are taken to the product details page to view more complete product information.

Scenario 2: The user selects an SKU and prepares to purchase.

After selecting a specific SKU on the product details page, such as "black" headphones, the page displays the corresponding price and stock for that SKU. After confirming, the user clicks "Buy Now".

Scenario 3: The user adds the item to their cart before making a purchase.

If a user is interested in a product but does not want to pay immediately, they can click "Add to Cart" and pay in the cart later.

Scenario 4: Product unavailable for purchase

If a product is no longer available or has zero stock, users should see a clear message when they enter the product details page, and the purchase button should be grayed out and unclickable.

4.4 Page Structure

4-1 产品详情页线框图

Product Detail Page Wireframe

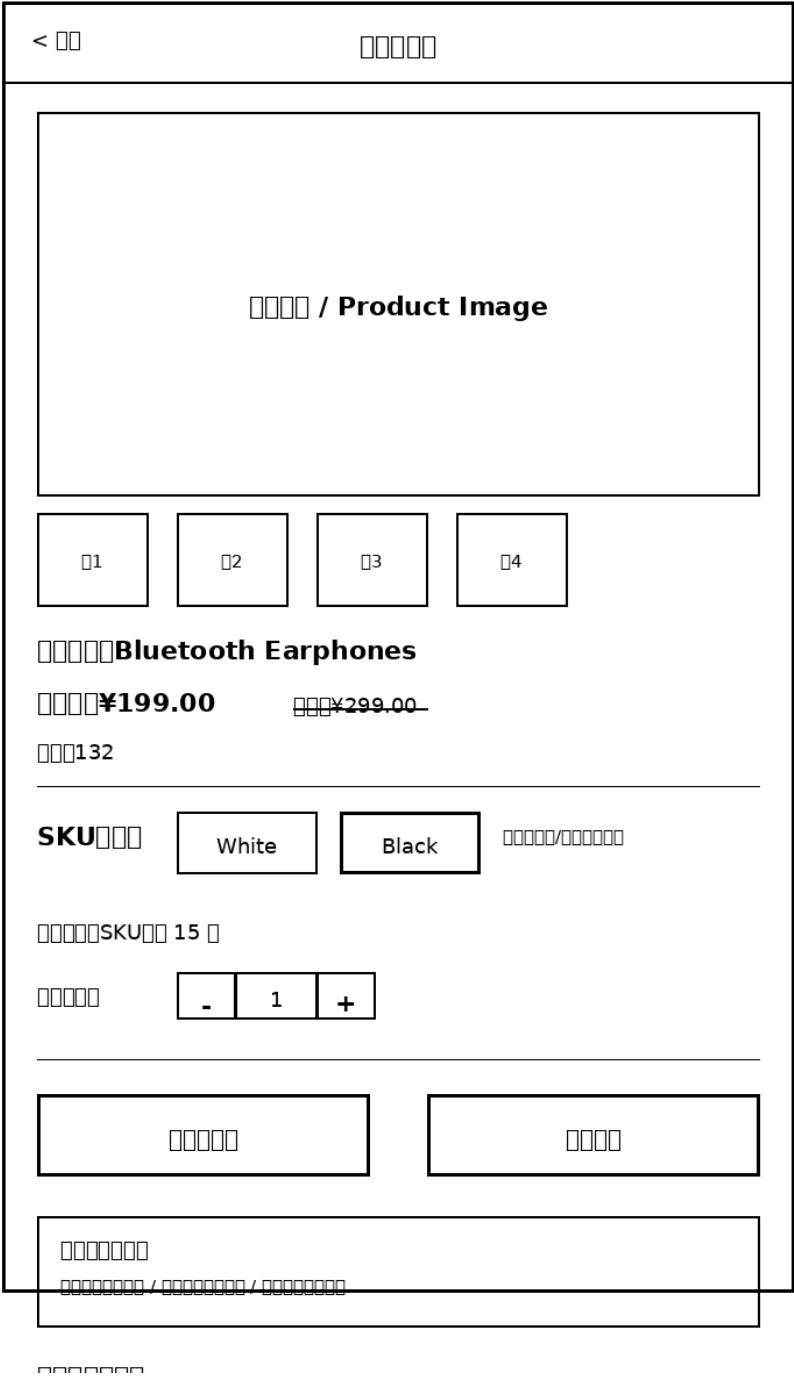


Figure 4-1 Low-fidelity prototype of the product details page

4.5 Page Field Design

area	Field	Field Description	Is it necessary?
Product Image Area	Product main image	Displaying key product images helps users quickly identify the product.	yes
Product Image Area	Product Image List	Displays multiple product images, allowing users to view more details.	no
Product Information Area	Product Name	Display product names, such as Bluetooth Earphones	yes
Product Information Area	Product ID	Used for system product identification; may not be displayed on the front end.	yes
Product Information Area	Product status	Determine whether a product is listed, for example, ON_SALE / OFF_SALE.	yes
Price Zone	Selling price	The actual price paid by the user for the goods	yes
Price Zone	Original price	Used for price comparison displays to help users understand discounts.	no
Sales data area	Sales	Displaying cumulative sales volume enhances user trust.	no
SKU area	SKU Name	Display specification options, such as white and black.	yes
SKU area	SKU Price	Different SKUs may have different prices.	yes
SKU area	SKU inventory	Current SKU available quantity	yes
Inventory Area	Total inventory	Overall inventory of goods, used to determine overall sell-out status.	yes
Quantity area	Purchase Quantity	The quantity the user plans to purchase this time	yes
Operating area	Add to cart button	Users add items to their shopping cart	yes
Operating area	Buy Now button	Users are directly taken to the order confirmation page.	yes

Prompt Area	Error message	Display abnormal information such as unavailable, sold out, and insufficient stock.
-------------	---------------	---

4.6 Core Interaction Design

4.6.1 Accessing the product details page

After a user clicks on a product from the product list, the system requests the product details API based on the product ID and displays the product information.

If the API returns a successful response, the page will display information such as product name, image, price, SKU, and inventory.

If the API fails, an error message will be displayed on the page.

Prompt message: Product information failed to load, please try again later.

4.6.2 SKU Selection

If a product has multiple SKUs, users need to select the SKU before adding it to their cart or purchasing it immediately.

For example:

Product: Bluetooth Earphones

SKU 1: White, Price: ¥199.00, Stock: 10

SKU 2: Black, Price ¥209.00, Stock 15

When a user selects a different SKU, the page needs to be updated accordingly:

1. Current displayed price;
2. Current SKU inventory;
3. Purchase button status;
4. The maximum number of items a user can purchase.

If a user clicks "Add to Cart" or "Buy Now" without selecting an SKU, the system should prompt the user to select the specifications first.

Prompt text: [Please select the product specifications first.](#)

4.6.3 Selection of Purchase Quantity

Users can adjust the purchase quantity using the quantity selector.

Purchase quantity rules:

1. The default purchase quantity is 1;
2. The quantity purchased must be greater than 0;
3. The quantity purchased cannot exceed the current SKU inventory;
4. If the current SKU inventory is 0, the quantity selector is not operable;
5. If the user enters an invalid quantity, the system should automatically correct it or display an error message.

Prompt text: [The quantity purchased cannot exceed the current inventory.](#)

4.6.4 Add to Cart

When a user clicks "Add to Cart," the system needs to verify:

1. Is the user already logged in?
2. Is the product currently listed?
3. Has the user selected an SKU?
4. Is the current SKU in stock?
5. Is the quantity purchased legal?
6. Once the verification is successful, the item will be added to your shopping cart.

Success message: [Added to cart.](#)

If the user is not logged in, you can guide them to log in first.

Prompt text: [Please log in before adding to your cart.](#)

4.6.5 Buy Now

When a user clicks "Buy Now", the system needs to verify:

Is the user already logged in?

1. Is the product currently listed?
2. Has the SKU been selected?
3. Are there sufficient SKUs in stock?
4. Is the quantity purchased legal?
5. After successful verification, the user is directed to the order confirmation page.

If the verification fails, the user will remain on the product details page with the corresponding error message displayed.

4.7 Business Rules

Rule 1: Products must be listed before they can be purchased.

When the product status is **ON_SALE** At that time, users can normally select SKUs, add items to their cart, and purchase immediately.

When the product status is **OFF_SALE** The goods cannot be purchased at that time.

Page presentation:

1. The product is now unavailable.
2. The "Add to Cart" button is grayed out.
3. The "Buy Now" button is grayed out;

Users are unable to submit purchase requests.

Notice: This product is no longer available and cannot be purchased at this time.

Rule 2: Users must select an SKU before they can make a purchase.

If a product has an SKU list, users must select a specific SKU before adding it to their cart or purchasing it immediately.

The reason is that orders need to be linked to specific SKUs to determine prices, inventory, and subsequent shipping specifications.

Prompt text: Please select the product specifications first.

Rule 3: The price on the page should change dynamically based on the SKU selected by the user.

Products may have a base price or an SKU price.

When a user does not select an SKU, the page can display the default product price or price range.

After a user selects a specific SKU, the page should display the price corresponding to that SKU.

For example:

User selects White SKU: Price displayed: ¥199.00; Stock displayed: 10.

User selects Black SKU: Price displayed: ¥209.00; Stock displayed: 15

Rule 4: Inventory display should be based on the current SKU inventory.

After a user selects an SKU, the page should display the current SKU inventory, rather than just the total inventory of the products.

The reason is that different SKUs may have different inventory levels. If only the total inventory is displayed, users may mistakenly believe that all specifications are in stock, leading to order failure.

Rule 5: You cannot purchase when inventory is 0.

When the total inventory of a product is 0, or when the inventory of the SKU selected by the user is 0, the purchase button should be grayed out and not allowed to be clicked by the user.

The page indicates that the current stock of this product is insufficient.

The button text can be changed to: [Sold out](#)

Rule Six: Adding items to your cart does not lock in inventory.

When a user adds an item to their shopping cart, the system should not immediately deduct the inventory.

The reason is that adding items to the shopping cart only indicates a user's intention to buy, and does not mean that the user will actually pay.

Inventory should be locked or deducted according to the system's inventory policy during the order submission or payment stage.

Rule 7: Product status and inventory must be re-verified before submitting an order.

The information displayed on the product details page may be cached or delayed, so the backend must re-validate when a user clicks "Buy Now" or submits an order from the shopping cart.

1. Is the product listed?
2. Is the SKU valid?
3. Has the price changed?
4. Is the inventory sufficient?
5. Is the quantity purchased legal?

If backend verification fails, the user should be prevented from submitting the order.

Rule 8: Price display requires conversion of the amount format.

The price in the API is usually returned in cents, for example: price = 19900

The front-end page should display: ¥199.00

It cannot be directly displayed as: ¥19900

Otherwise, it will cause a serious price display error. Once this error goes live, it will give the product manager a taste of what it means to be "publicly executed."

4.8 Exception Handling and Error Messages

Abnormal scenarios	Triggering conditions	Page performance	Prompt text
Product information failed to load	Product details API request failed	The page displays a loading failure status.	Product information failed to load, please try again later.
The product has been removed from the shelves.	The product status is OFF_SALE	Gray out the purchase button	This product has been removed from the shelves and is temporarily unavailable for purchase.
Sold out	Total inventory is 0	The "Buy" button is grayed out, indicating that the product is sold out.	The product is currently sold out.
SKU inventory insufficient	The user selected SKU with 0 inventory.	SKU is currently unavailable for purchase.	Current specifications are in short supply.
No SKU selected	The user clicked to buy without selecting an SKU.	Prevent operation	Please select the product specifications first.
Purchase quantity exceeds inventory	buyCount > skuStock	Prevent operation	The quantity purchased cannot exceed the current inventory.
User not logged in	Users who are not logged in can click "Add to Cart" or "Buy Now".	Guide to Login	Please log in before proceeding.

Price changes	Prices differ before and after placing the order	Prevent ordering at the original price and prompt to refresh.	The product price has been updated, please confirm again.
Network error	Request timed out or network failed	Keep the current page open and allow retries.	Network error, please try again later.

4.9 Acceptance Criteria

4.9.1 Page Display Acceptance

1. After entering the product details page, users can see the product name, product image, price, sales volume, SKU, and inventory information.
2. Product images load correctly.
3. The product price is displayed in the correct format, for example, ¥199.00.
4. If an original price exists, the page should display that price.
5. If sales figures exist, the page should display those figures.

4.9.2 SKU Interactive Acceptance

1. The page can display all available SKUs.
2. When a user selects different SKUs, the price on the page changes correctly.
3. After a user selects different SKUs, the inventory on the page updates correctly.
4. SKUs with zero stock cannot be purchased.
5. If a user clicks to buy without selecting an SKU, the system prompts "Please select product specifications first".

4.9.3 Inventory Rule Acceptance

1. When the total inventory of goods is 0, the purchase button is grayed out.
2. When the current SKU inventory is 0, the purchase button is grayed out.
3. The system blocks the operation when a user purchases more than the stock is available.
4. The inventory quantity displayed on the page is consistent with the inventory returned by the API.

5. Before submitting an order, the system will re-verify the inventory.

4.9.4 Goods Condition Inspection

1. When the product status is ON_SALE, users can make a purchase normally.
2. When the product status is OFF_SALE, the page displays "Product is no longer available".
3. The "Add to Cart" and "Buy Now" buttons for OFF_SALE items are not clickable.
4. If a product is removed from the shopping cart, the user cannot submit an order for that product at checkout.

4.9.5 Add to cart and check

1. After selecting the SKU and quantity, the user can successfully add the item to their shopping cart.
2. After successfully adding the item to your cart, a success message will be displayed on the page.
3. When an unlogged-in user clicks "Add to Cart," the system will guide them to log in.
4. You cannot add items to your cart if they are unavailable or out of stock.

4.9.6 Immediate Purchase and Acceptance

1. After selecting the SKU and quantity, users can click "Buy Now".
2. After clicking "Buy Now", the system will redirect you to the order confirmation page.
3. You cannot proceed to the order confirmation page if the product is unavailable, out of stock, or if no SKU is selected.
4. Before redirecting to the order confirmation page, the system should verify the product status, SKU status, inventory, and price.

4.10 Data Tracking and Recording

Although the question does not mandate the use of tracking data, from a product manager's perspective, the product details page is a crucial node in the conversion process, so it is recommended to record the following basic data.

Event tracking	Triggering timing	use
product_detail_view	User enters product details page	Statistics on product exposure and page views
sku_select	User selects SKU	Specifications for analyzing user preferences
add_to_cart_click	The user clicks "Add to Cart"	Analysis of purchase intentions
buy_now_click	The user clicked "Buy Now"	Analyze purchase intentions
add_to_cart_success	Added to cart successfully	Statistics on the success rate of adding items to cart
buy_now_success	Successfully entered the order confirmation page	Analyze the conversion rate from the product details page to the order page.
product_unavailable_show	Display notifications for items being unavailable or sold out.	Monitoring the impact of unpurchaseable items on user experience

This data can help in subsequent analysis of product detail page views, add-to-cart rate, purchase conversion rate, and the impact of inventory anomalies on conversion.

4.11 Relationship with other modules

The product details page is not an isolated function; it needs to work in conjunction with multiple modules.

Related modules	Relationship Explanation
Product Management Module	The backend maintains information such as product name, image, price, and status, while the frontend product details page reads and displays this information.
SKU Management Module	The product details page displays SKU options, and shows prices and stock levels based on the SKU.
Inventory Management Module	The product details page displays the inventory; inventory verification is required before placing an order.

Shopping cart module	After the user clicks "Add to Cart," the product, SKU, and quantity are added to the cart.
Order module	After the user clicks "Buy Now", they will proceed to the order confirmation and order submission process.
User Module	Check if the user is logged in; if not, the user needs to log in first.
Payment module	The product details page does not directly process payment, but it guides users through the subsequent order payment process.

4.12 Scope of this MVP and subsequent iterations

MVP Implementation Scope for This Period

This product details page primarily implements the following capabilities:

1. Display basic product information;
2. Display product images;
3. Displays the selling price and original price of the product;
4. Display SKU list;
5. Supports SKU selection;
6. Display prices and inventory based on SKU;
7. Add to cart;
8. Supports immediate purchase;
9. Supports product delisting and low inventory alerts.

Subsequent iterative directions

Subsequent versions may add the following capabilities:

1. Product reviews are displayed.
2. Rich text description of product details;
3. Product parameter table;
4. Product recommendations;
5. Coupon redemption;
6. Promotional activity labels;
7. Collect items;

8. Share products;
9. Arrival notification;
10. Recommended similar products.

These features will not be built during the MVP phase because they do not affect the basic transaction loop and would increase the complexity of front-end display, back-end configuration, and business rules.

V. API Comprehension Questions

This section analyzes the product detail page API response provided by the engineering team, explaining what information the front-end product detail page can display, how the price field should be handled, and how the page should respond in scenarios such as product listing/delisting, inventory changes, and SKU selection. Additionally, this section will supplement the analysis from a product manager's perspective, highlighting any key fields currently missing from the API.

5.1 Based on this API response, what information can be displayed on the product details page?

According to the API response, the product details page can display basic product information, price information, inventory information, sales volume information, product images, and SKU information.

First, the page can display basic product information.

in, `productId = 10001` It is a unique product ID, mainly used for internal system identification, API calls, data tracking and order association, and is not usually directly displayed to ordinary users. `productName = "Bluetooth Earphones"` It can be used as the product title displayed on the product details page. `status = "ON_SALE"` This indicates that the product is currently available for sale, and users can browse and make a purchase.

Secondly, the page can display product price information.

The API returned `price = 19900` and `originalPrice = 29900` Here, "price" can be understood as the current selling price, and "originalPrice" can be understood as the original price or the crossed-out price. The

page can highlight the current selling price while displaying the original price in a weaker style to express discount or promotional information.

Third, the page can display inventory and sales information.

`stock = 25` This indicates that the current total inventory of the product is 25 units. The page can display this as "Inventory 25 units" or it can be used to determine whether the purchase button is clickable. `salesCount = 132` This indicates that the product has sold a total of 132 units, which can be displayed as "132 units sold" to help users determine the product's popularity.

Fourth, the page can display product images.

`images` The field returns the URL of the product image, which the front-end can use as the main product image. If multiple images are returned in the future, this can be expanded to display a product carousel or detail images.

Fifth, the page can display SKU specification information.

`skuList` The field contains product specifications for different sizes, such as White and Black. Each SKU has its own...`skull`、`skuName`、`price` and `stock` Therefore, the page can display color options and dynamically update prices and inventory as the user selects different SKUs.

Overall, the API is already able to support the core display of a basic product details page, including product name, product image, current price, original price, inventory, sales volume, product status, and SKU selection.

5.2 How should the price of 19900 be displayed to the user?

`price = 19900` It should not be displayed as "19900". In e-commerce systems, price fields are usually returned in "cents" rather than directly in "yuan".

Therefore, the front end needs to `19900` Divide by 100 and display to the user:

`19900 / 100 = 199.00`

Therefore, the page should display as follows: **¥199.00**

Similarly, `originalPrice = 29900` It should be displayed as: **¥299.00**

When displaying the price on the page, the current price can use a more prominent style, such as bold or highlighted; the original price can be displayed using a gray underlined price. A reasonable display method is:

Current price: ¥199.00

Original Price: ¥299.00

This approach aligns with users' perception of price while avoiding misunderstandings about units that could lead to incorrect price displays. Otherwise, displaying a Bluetooth headset as ¥19,900 might make users think it can also help them find a job—outrageous, but not entirely without market potential.

5.3 If status = OFF_SALE, how should the page handle this?

if status = OFF_SALE This indicates that the product has been removed from the shelves and users should not be allowed to continue purchasing it.

The page can continue to display basic product information, but the purchase area needs to clearly indicate: "This product is no longer available and cannot be purchased at this time."

Meanwhile, the "Add to Cart" and "Buy Now" buttons should be grayed out and disabled so that users cannot click them to proceed to the order process. This can prevent users from submitting invalid orders and reduce the pressure on the backend for error checking.

If a user accesses the product details page from their shopping cart, favorites, order history, or an external link, the page should still be accessible, but the "out of stock" status must be clearly displayed. This allows users to understand why the product is unavailable for purchase, rather than seeing a blank page or a system error.

In addition, the page can also provide entry points such as "View similar products" or "Return to product list" to guide users to continue browsing other products and reduce user churn.

therefore, OFF_SALE The core principles for handling product status are: products can be displayed but cannot be purchased; the page must clearly indicate the product status; and all purchase-related operations must be disabled.

5.4 How should the "Buy" button be displayed if the total inventory of the product is 0?

If the total inventory of goods `stock = 0` This indicates that the product is currently out of stock. In this case, the "Add to Cart" and "Buy Now" buttons should be grayed out and disabled.

The button text can be changed to:

Sold out

or:

Insufficient inventory

At the same time, the inventory area should display a clear message, such as: "This item is currently out of stock."

The quantity selector should also be disabled, or users should be prevented from increasing the purchase quantity.

It should be noted that even if the product status is still...`ON_SALE` If the inventory is zero, a user should not be able to submit an order. Product status indicates whether a product is available for sale, while inventory determines whether a product is actually currently available for purchase.

If the system supports restocking alerts in the future, a "Refresh Arrival" button can be displayed in this scenario. However, for the MVP version, restocking alerts are not a necessary feature to complete the basic transaction loop and can be considered as a feature for future optimization.

5.5 If a user selects the Black SKU, how should the page price and inventory change?

If the user selects the Black SKU, the page should display accordingly. `skuList` The SKU information for Black is dynamically updated with prices and inventory.

The information for the Black SKU is as follows:

`skuId = 20002`

`skuName = "Black"`

`price = 20900`

`stock = 15`

Therefore, when a user selects the Black SKU, the price displayed on the page should be updated to: **¥209.00**

The inventory should be updated to: **15 items in stock**

At the same time, the corresponding Black SKU should also be passed when submitting the order. `skuId = 20002` Instead of just passing the product ID, it should record the product ID. This is because different SKUs of the same product may have different prices, stock levels, colors, or specifications. If the order only records the product ID, it's impossible to accurately determine whether the user purchased White or Black, and it's also impossible to correctly deduct the corresponding SKU's stock.

Therefore, the page interaction logic after SKU selection can be described as follows:

The user selects the Black SKU → the page price is updated to ¥209.00 → the page inventory is updated to 15 pieces → when submitting the order, the `skuId = 20002` is passed → the backend verifies the price and inventory based on this SKU.

If a certain SKU has 0 units in stock, it should be displayed as unavailable for purchase, and the user should not be able to select that specification to submit an order.

5.6 What fields are missing from this API response?

While the current API already supports the basic display of product detail pages, from a product manager's perspective, it still lacks some key fields. It is recommended to add the following fields:

1. Product details description field: `productDescription`

The current API only returned the product name and image, but not detailed product information. A product details page typically needs to display product selling points, features, applicable scenarios, materials, dimensions, after-sales service, and other information.

Without detailed product descriptions, users can only see the product name and price, and cannot fully understand the product's value, which can affect their purchasing decisions.

2. Product category fields: `categoryId` / `categoryName`

The API lacks product category information. Product categories help the system determine which category a product belongs to, such as "digital products," "headphones," or "Bluetooth devices."

This field can be used not only for page display, but also for product recommendations, search filtering, operations management, and data analysis.

3. SKU Attribute Field: skuAttributes

Currently, there are only SKUs.`skuName`For example, White and Black, but it is not clearly stated whether the SKU corresponds to color, size or other attributes.

A more reasonable structure should include SKU attributes, for example:

Color: Black

Size: Standard

Version: Basic

This allows the front-end to display the specification options more clearly and also makes it easier to expand to multiple specifications of products in the future.

4. Product listing/delisting time fields: saleStartTime / saleEndTime

The current API only returns the product status, but does not return the product's listing time or delisting time.

If a product involves scheduled listings, limited-time sales, pre-sales, or promotional activities, these time fields are needed to support page display and business logic. For example, the page can display information such as "Sale Coming Soon" or "Remaining Time for the Event".

5. Shipping and delivery information fields: shippingFee / deliveryInfo

Product detail pages typically need to display information such as shipping method, estimated delivery time, and shipping fees. The current API does not return these fields.

For users, product price is only one part of the purchasing decision; shipping costs and delivery time also influence whether or not they place an order. For example, given two products with the same price, one with free shipping and the other with very high shipping costs, users may make completely different choices.

6. After-sales policy field: returnPolicy / refundPolicy

The product details page should also display return and exchange policies, refund rules, and after-sales instructions. Especially in e-commerce systems, users typically care about whether a seven-day no-reason return policy is supported, whether refunds are available, and whether a warranty is offered before making a purchase.

Without after-sales policies, users may lose trust in the product, and customer service will face increased pressure in responding to inquiries.

7. Product status display text field: statusText

The current API only returned `status = ON_SALE`. This is a more technical status value. The front-end can automatically convert it to "available for sale," but if different statuses correspond to complex text, it's best for the back-end or configuration system to return the corresponding display text.

For example:

ON_SALE: Available for sale

OFF_SALE: No longer available.

SOLD_OUT: Sold out

COMING_SOON: Coming Soon

This reduces hard coding on the front end and makes it easier to maintain status text uniformly in the future.

In summary, the recommended core fields to add include: product details description, product category, SKU attributes, listing and delisting times, shipping information, after-sales policy, and status display text. These fields can help upgrade the product details page from "displaying basic product information" to "supporting users' complete purchase decisions."

VI. Simple Code Logic Reading Questions

This section primarily analyzes the business logic of the pseudocode provided in the question. The core function of this feature is to determine whether a user can submit an order given the current product status, SKU inventory, and purchase quantity. Although the code itself is relatively simple, it embodies a crucial backend validation logic in e-commerce systems: the frontend can guide user actions, but the final decision on whether to allow an order must be made by the backend based on the product status and inventory.

The pseudocode in the problem is as follows:

```
function canSubmitOrder(productStatus, skuStock, buyCount) {  
  if (productStatus !== 'ON_SALE') {  
    return {  
      canSubmit: false,  
    }  
  }  
}
```

```

reason: 'Product is off sale'
};
}
if (skuStock <= 0) {
return {
canSubmit: false,
reason: 'Insufficient inventory'
};
}
if (buyCount > skuStock) {
return {
canSubmit: false,
reason: 'Purchase quantity exceeds available inventory'
};
}
return {
canSubmit: true,
reason: ""
};
}

```

6.1 What is this function primarily used to determine?

This function is mainly used to determine whether a user can submit an order.

Specifically, it makes a judgment based on three key conditions:

First, is the product available for sale?

If the product is not **ON_SALE** This indicates that the product may have been removed from the shelves, suspended from sale, or is unavailable for purchase. In this case, the user cannot submit an order.

Second, is the selected SKU in stock?

if **skuStock <= 0** This indicates that there is currently no available stock for this SKU, and the user cannot submit an order.

Third, whether the number of items purchased by the user exceeds the current SKU inventory.

If the user wants to buy the quantity `buyCount` Greater than the current SKU inventory `skuStock` This indicates that the inventory is insufficient to fulfill the order, and the user cannot submit the order.

The system will only allow users to submit an order if the product is available for sale, the SKU is in stock, and the purchase quantity does not exceed the stock limit.

From a product perspective, this function can be understood as the basic validation rules before order submission. Its purpose is to prevent users from purchasing items that are no longer available, out-of-stock items, or items in quantities exceeding the available stock.

6.2 Under what circumstances can a user not submit an order?

According to the function logic, users cannot submit orders in the following three situations.

1. The product is not currently for sale.

when `productStatus !== 'ON_SALE'` When the function returns:

```
{  
  canSubmit: false,  
  reason: 'Product is off sale'  
}
```

This means the product is not currently available for sale, and the user cannot submit an order.

The corresponding business scenarios include:

- The product has been removed from the shelves.
- The product was temporarily shut down by operations staff;
- The product has not yet been officially launched.
- The product has been banned from sale by the platform due to abnormal reasons.

In these situations, even if the user has already added the items to their shopping cart, they cannot proceed with the order submission. The system should prompt the user:

"The product is no longer available and cannot be purchased."

2. SKU inventory is less than or equal to 0.

when `skuStock <= 0` When the function returns:

```
{  
  canSubmit: false,
```

```
reason: 'Insufficient inventory'
}
```

This means that the SKU selected by the user is currently out of stock and the order cannot be submitted. For example, the product as a whole may still be available for sale, but a specific color or size selected by the user may be sold out. For instance, white headphones may still be in stock, but black headphones may be out of stock; in that case, the user cannot place an order for the black SKU.

The page should prompt the user: **"Insufficient stock, please select other specifications."**

3. The number of items purchased by the user exceeds the available inventory for the SKU.

when `buyCount > skuStock` When the function returns:

```
{
  canSubmit: false,
  reason: 'Purchase quantity exceeds available inventory'
}
```

This means that the quantity the user wants to buy exceeds the current available inventory.

For example, if there are currently 3 SKUs in stock, but a user wants to buy 5, the system cannot allow the user to submit an order because the stock cannot meet the order demand.

The page should prompt the user:

"The quantity purchased exceeds the available inventory. Please modify the purchase quantity."

6.3 If `productStatus = 'ON_SALE'`, `skuStock = 3`, and `buyCount = 5`, what result should the function return?

In this case:

```
productStatus = 'ON_SALE'
skuStock = 3
buyCount = 5
```

First, the product status is `ON_SALE`. This indicates that the product is available for sale and therefore passes the first layer of verification.

Secondly `skuStock = 3`. Since the inventory is greater than 0, it can pass the second-level verification.

but, `buyCount = 5` A user wants to buy 5 items, but the current stock of SKU is only 3. Since the purchase quantity exceeds the stock, the following applies:

`buyCount > skuStock`

Therefore, the function will enter the third condition and return the result that the order cannot be submitted.

The function should return:

```
{  
  canSubmit: false,  
  reason: 'Purchase quantity exceeds available inventory'  
}
```

The corresponding business meaning is:

Although the product is available for sale and the SKU is currently in stock, the user's purchase quantity exceeds the available stock, therefore the order cannot be submitted. The system should prompt the user to reduce the purchase quantity, for example:

"Only 3 items remain in stock. Please modify the quantity before submitting your order."

This scenario is very typical because seeing "in stock" doesn't mean a user can buy any quantity. If there are only 3 items in stock but the user wants to buy 5, the system will obviously stop them. Otherwise, warehouse staff would have to perform a performance of creating something out of nothing, adding another tragedy to human commercial civilization.

6.4 What additional checks should be added?

The current function only verifies product status, SKU inventory, and purchase quantity, covering the most basic order restrictions. However, in a real e-commerce system, submitting an order requires additional business validations to prevent abnormal orders and transaction risks. It is recommended to add the following validations:

1. User login status verification

Before submitting an order, the system needs to determine whether the user is already logged in.

If a user is not logged in, order submission should be prevented, and the user should be guided to log in first. This is because orders need to be linked to a user ID for subsequent order viewing, payment, refunds, after-sales service, and logistics tracking.

The corresponding rules could be:

If a user is not logged in, they cannot submit an order.

Prompt text: [Please log in before submitting your order.](#)

2. Delivery address verification

Users must select a valid shipping address before submitting an order.

If a user does not fill in the recipient's name, mobile phone number, or detailed address, or if the address is outside the delivery area, the system should prevent the order from being submitted.

The corresponding rules could be:

If the shipping address is empty or invalid, the order cannot be submitted.

Prompt text: [Please fill in a valid shipping address.](#)

3. Verification of the legality of the purchase quantity

The current code only checks `buyCount > skuStock`. However, it should also be determined whether the quantity purchased is a legal number.

For example:

The quantity purchased must not be less than 1;

The quantity purchased cannot be 0;

The quantity purchased cannot be negative;

The quantity purchased cannot be a decimal;

The quantity to be purchased cannot be a non-numeric character.

Otherwise, the user may submit `buyCount = 0` or `buyCount = -1`. This kind of abnormal data. Although normal users wouldn't do this, there are always humans and scripts that like to push the boundaries of the system, forcing products and technologies to mature.

The corresponding rules could be:

If the quantity purchased is less than 1 or is not a positive integer, the order cannot be submitted.

Prompt text: ["Please enter a valid purchase quantity."](#)

4. Individual Purchase Limit Verification

Some products may have purchase limits, such as a maximum of 2 items per user.

Therefore, the system needs to determine whether the quantity purchased by the user this time plus the quantity purchased in the past exceeds the purchase limit.

The corresponding rules could be:

If a user purchases more than the limit, the order cannot be submitted.

Prompt text: ["This product is limited to 2 items per person."](#)

This rule is very common for promotional items, scarce items, and platform-subsidized items, and can prevent malicious hoarding or order manipulation.

5. Product Price Verification

When a user submits an order, the system should re-verify the product price and should not completely trust the price transmitted from the front end.

Because product prices may change after a user browses the product details page, or may be modified by the front-end, the back-end should use the real-time price in the database.

The corresponding rules could be:

If the price passed from the front end is inconsistent with the current price on the back end, the price on the back end should prevail, and the user should be notified of the price change if necessary.

Prompt text: "The product price has been updated. Please confirm and resubmit your order."

6. Is the product still subject to verification?

In addition to determining whether a product is for sale, it is also necessary to determine whether the product itself still exists.

For example, the product may have been deleted by operations staff, or the product ID may be invalid. In this case, the system should prevent order submission.

The corresponding rule could be: if the product does not exist, the order cannot be submitted.

Prompt text: "The product is unavailable or expired."

7. SKU validity verification

When a user submits an order, the system needs to determine whether the selected SKU still belongs to the product and whether the SKU is valid.

For example, the SKU selected by the user may have been deleted, or an incorrect SKU may have been passed in from the front end. Failure to verify may result in incorrect product specifications in orders, or even affect inventory deductions.

The corresponding rules could be:

If the SKU does not exist or is not part of the current product, the order cannot be submitted.

Prompt text: "Product specifications are invalid. Please select again."

8. Validation of items in the shopping cart

If a user submits an order from their shopping cart, the system also needs to check whether the items in the shopping cart are still valid.

Possible anomalies include:

The product has been removed from the shelves.

- SKU is no longer valid;
- Insufficient inventory;
- Prices have changed;
- The product has been removed.

The corresponding rules could be:

If the items in your shopping cart are expired, you cannot submit your order.

Prompt text: "There are invalid items in your shopping cart. Please modify them and submit again."

9. Payment Method Verification

If a payment method is required when submitting an order, the system should determine whether the payment method is available.

For example, some payment methods may be temporarily under maintenance, or the current order amount may not support a certain payment method.

The corresponding rules could be:

If the payment method is unavailable, you cannot submit your order.

Prompt text: "The current payment method is unavailable. Please select another payment method."

10. Risk control verification

In real e-commerce systems, submitting an order may also require basic risk control assessments.

For example:

- Is the user account abnormal?
- Does the company engage in frequent order placement?
- Is there a risk of malicious order placement?
- Is the delivery address or phone number abnormal?
- Is the order amount abnormal?

If risk control rules are triggered, the system can restrict order submissions or require further verification.

The corresponding rules could be:

If an order triggers risk control rules, the order cannot be submitted temporarily.

Prompt text: "The order has an error. Please try again later or contact customer service."

6.5 Summary

Overall, the business logic of this function is as follows: before a user submits an order, the system needs to verify whether the product is available for sale, whether the SKU is in stock, and whether the purchase quantity exceeds the stock. Only when all conditions are met can the user submit the order.

The current function covers the most basic inventory and product status verification, but real e-commerce systems also need to supplement verification for user login, shipping address, purchase quantity validity, purchase limit rules, price changes, SKU validity, shopping cart product status, payment method, and risk control.

Product managers need to understand the business meaning behind each condition and be able to translate code logic into clear page prompts, interaction rules, and exception handling solutions.

VII. Order Status Comprehension Questions

This section primarily analyzes the order status design in e-commerce systems. Order status is one of the core fields in the e-commerce transaction process, determining what actions users, operations personnel, and the system can perform at different stages. For example, it determines whether a user can pay after submitting an order, whether a refund can be requested after payment, whether the merchant can ship the goods, and whether the order is ultimately completed.

The order status given in the question includes:

- **PENDING_PAYMENT**Pending payment
- **PAID**Paid
- **SHIPPED**Shipped
- **COMPLETED**Completed
- **CANCELLED**Cancelled
- **REFUNDING**Refund in progress
- **REFUNDED**Refund has been issued.

These statuses cover the main order lifecycle in a basic e-commerce system, including normal transaction processes and abnormal refund processes.

7.1 What should be the initial status after a customer submits an order?

After a customer submits an order, the initial status of the order should be:

PENDING_PAYMENTPending payment

The reason is that a user submitting an order only means that the order has been successfully created, but it does not mean that the transaction is complete. At this time, the user has not yet completed payment, so the system cannot directly mark the order as "paid" or "completed".

At this stage, the system should generate an order number and lock the corresponding product inventory, waiting for the user to complete the payment within the specified time.

For example, after a user clicks "Submit Order," the system can generate an order record with a status of...**PENDING_PAYMENT**The page can display:

"Order has been submitted. Please complete payment within 15 minutes."

In this state, users can typically perform the following operations:

- Continue making payments;
- Cancel the order;
- Return to the order list;
- View order details.

The system also needs to be configured with a payment countdown. If the user fails to complete payment within the specified time, the order should be automatically cancelled, and the locked inventory should be released.

Therefore, the initial status after submitting an order should be:**PENDING_PAYMENT**The order has been created, but payment has not yet been completed.

7.2 What should the order status be after successful payment?

After successful payment, the order status should change from:

PENDING_PAYMENTPending payment

It becomes:

PAIDPaid

The reason is that the user has completed the payment, the system has confirmed receipt of the payment result, and the order has now entered the pending shipment stage.

At this stage, users can no longer cancel orders at will. Operations staff or merchants need to see the order in the backend and arrange the subsequent delivery process.

State changes can be described as:

User submits order → Order status is **PENDING_PAYMENT**

User completes payment → Payment system returns success result → Order status updated to **PAID**

When the order status changes **PAID** Then, the system can trigger the following operations:

1. Notify the user that payment was successful;
2. Notify operations staff to process the order;
3. Generate a task to be shipped;
4. Update order payment time;
5. Record the payment transaction number;
6. Ready to proceed with the shipping process.

The page can display:

"Payment successful, awaiting shipment from the merchant."

It's important to note that order status updates must be based on the backend payment callback result, not just the frontend page display. This is because a user seeing a payment success page doesn't necessarily mean the backend has actually received a payment success notification. E-commerce systems can't rely on "the user saying they've paid" to ship goods; otherwise, the platform would quickly become a charity, and the kind that goes bankrupt.

7.3 What should the order status be after the merchant ships the goods?

After the merchant ships the goods, the order status should change from:

PAIDPaid

It becomes:

SHIPPEDShipped

The reason is that the order has already been paid for and the merchant has shipped the goods. At this point, the order has entered the logistics and delivery stage.

State changes can be described as:

Order payment successful → Status: **PAID**

The operations staff fills in the logistics information and confirms the shipment → Status updated to **SHIPPED**

exist **SHIPPED** In this state, the system typically needs to record the following information:

1. Delivery time;
2. Logistics company;
3. Tracking number;
4. Delivery status;
5. Delivery address;
6. Estimated delivery time.

The client page can display:

"The merchant has shipped the goods, and they are being delivered."

It also provides a logistics tracking portal, for example: "check logistics"

For operations staff, the SHIPPED status indicates that the order has been shipped and should no longer appear in the "Orders Pending Shipment" list, but should instead be moved to the "Shipped Orders" or "Orders in Delivery" list.

7.4 What should the order status be if the customer fails to make payment within the payment deadline?

If the customer fails to complete payment within the payment deadline, the order status should change from:

PENDING_PAYMENT Pending payment

It becomes:

CANCELLED Cancelled

The reason is that although the order has been created, the user has not paid within the specified time. To prevent invalid orders from holding inventory for an extended period, the system needs to automatically cancel the orders and release the locked inventory.

For example, the system is set to allow 15 minutes for payment to be valid. If a user does not pay within 15 minutes of submitting an order, the system should automatically perform the following actions:

Update the order status to **CANCELLED**;

Release the inventory held by this order;

Record the reason for cancellation, for example "Payment timed out and automatically cancelled";

The cancellation status will be displayed on the order details page;

Notify users that their orders have been cancelled if necessary.

The page can display:

"The order has been automatically cancelled due to payment timeout."

It's important to note that both payment timeout cancellation and user-initiated cancellation can be accessed. **CANCELLED** Status, but the reason for cancellation should be recorded separately. For example:

User-initiated cancellation: **User cancelled**

Payment timeout cancellation: **Payment timeout**

Cancel in the background: **Admin cancelled**

This way, when conducting subsequent data analysis, we can identify the main reasons for order cancellations, instead of just seeing a bunch of cold, hard data. **CANCELLED** It's like a product manager's tombstone.

7.5 Under what circumstances should a customer be allowed to request a refund?

In a basic e-commerce system, whether a customer can apply for a refund depends on the current status of the order and the business rules.

Generally, customers are allowed to request a refund in the following situations:

1. PAID: Paid, not shipped

Users should typically be able to request a refund when an order has been paid for but the merchant has not yet shipped the goods.

This is the simplest refund scenario because the goods haven't been shipped yet, and there's no logistics interception or return processing involved. The system can change the order status from... **PAID** Updated to **REFUNDING** Updated after successful refund **REFUNDED**.

State transitions can be:

PAID → **REFUNDING** → **REFUNDED**

The page can display the following message:

"Refund request has been submitted, please wait for processing."

2. SHIPPED: Shipped

Whether a user can apply for a refund after the order has been shipped depends on the platform's rules.

In many e-commerce scenarios, refunds or returns can still be requested for shipped orders, but the process is more complicated. This is because the goods have already entered the logistics stage, and users may need to refuse delivery, return the goods, or wait for the merchant's approval.

State transitions can be:

SHIPPED → REFUNDING → REFUNDED

However, in this case, the system may need to add fields such as return logistics information, merchant review status, and refund reason.

3. COMPLETED: Completed

Whether a refund is allowed after an order is completed also depends on the after-sales policy.

If the platform supports after-sales refunds, such as a 7-day no-reason return policy, thenCOMPLETEDThe system also allows users to apply for refunds or after-sales service.

State transitions can be:

COMPLETED → REFUNDING → REFUNDED

However, certain conditions must be met, for example:

- Still within the after-sales period;
- The product supports returns and exchanges;
- The user provides the reason for the refund;
- The goods are in good condition for resale.
- Merchant approved.

4. It is not recommended to allow refunds.

The following situations generally should prevent users from requesting a refund again:

PENDING_PAYMENTPending payment

Since the user hasn't paid yet, there's no refund issue. If the user doesn't want to buy, they can cancel the order.

CANCELLEDCancelled

If the order has been cancelled and payment was not made, a refund is not required; if payment was made and the order was cancelled, the refund process should begin instead of remaining in the normal cancellation status.

REFUNDING Refund in progress

The order is already being processed for a refund; you should not submit a refund request again.

REFUNDED Refund has been issued.

The refund has been completed and you should not request a refund again.

Therefore, a more reasonable design is:

Refund application status: **PAID**、**SHIPPED**、**COMPLETED**

Refund requests are not permitted. **PENDING_PAYMENT**、**CANCELLED**、**REFUNDING**、**REFUNDED** in, **PAID** The simplest way to get a refund is under the current status; **SHIPPED** and **COMPLETED**

Refunds in this status need to be determined by considering return policies, after-sales service policies, and review rules.

7.6 Order Status Transition Process Description

The order status flow of a basic e-commerce system can be divided into normal transaction process, payment timeout cancellation process, and refund process.

1. Normal transaction process

Under normal circumstances, orders will flow through the following path:

PENDING_PAYMENT → **PAID** → **SHIPPED** → **COMPLETED**

The specific meanings are as follows:

After the user submits the order, the order enters... **PENDING_PAYMENT** Status: Awaiting user payment.

After the user successfully pays, the order enters... **PAID** Status: Awaiting shipment from the seller.

After the merchant confirms shipment, the order enters... **SHIPPED** Status: Waiting for user to receive the goods.

After the user confirms receipt or the system automatically confirms receipt, the order enters... **COMPLETED** Status: Transaction completed.

This process is the core transaction loop of an e-commerce system.

2. Payment timeout or user cancellation process

If a user submits an order but does not complete payment, the order can be withdrawn from [the payment process]. **PENDING_PAYMENT** The state becomes **CANCELLED**.

The state transition is as follows:

PENDING_PAYMENT → CANCELLED

Triggering scenarios include:

- The user canceled the order.
- The user has not made payment within the payment deadline;
- The system detected an order anomaly and cancelled it.
- The operations staff canceled the order in the backend.

In this process, if an order has not yet been paid, the inventory should be released upon cancellation, but a refund is not required.

3. Refund Process

If the user has already paid for the order, the refund process can be initiated based on the order status.

The typical refund status progression is as follows:

PAID → REFUNDING → REFUNDED

This applies to orders that have been paid for but not yet shipped.

If the order has already been shipped, it may also enter the refund process:

SHIPPED → REFUNDING → REFUNDED

This applies to scenarios where users apply for returns and refunds, refuse to accept goods, or the merchant agrees to a refund.

If the order has been completed but is still within the after-sales period, it may also enter the refund process:

COMPLETED → REFUNDING → REFUNDED

This applies to scenarios where the platform supports after-sales refunds.

in:

REFUNDING This indicates that the refund request has been submitted and is awaiting review or processing.

REFUNDEDThis indicates that the refund has been completed and the order amount has been returned to the user's account.

7.7 Order Status Transition Diagram

订单状态流转图

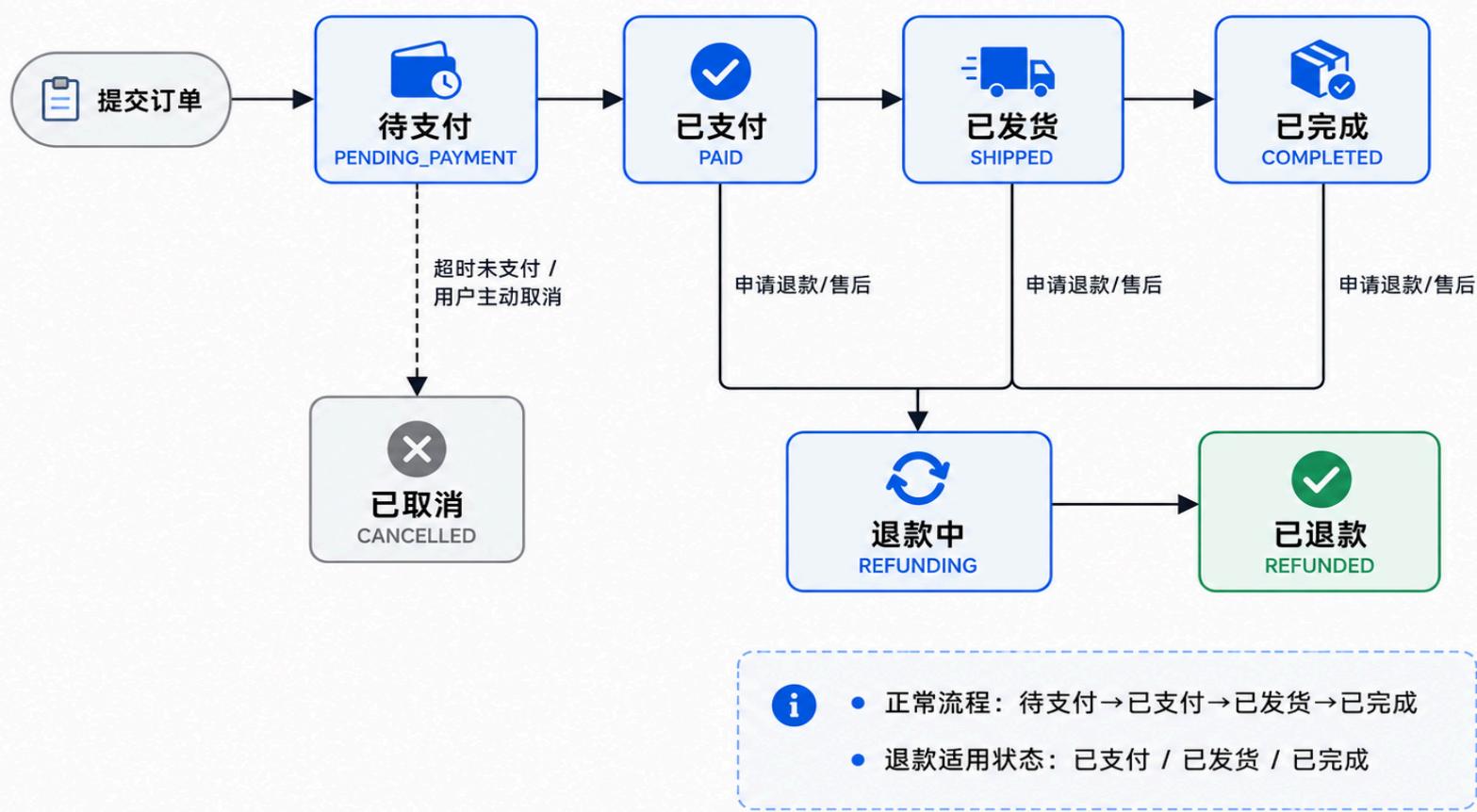


Figure 7-6 Order Status Transition Diagram

7.8 Considerations for Order Status Design

When designing order statuses, the following points should be noted:

First, order statuses should be mutually exclusive and clearly defined.

At any given time, an order should have only one primary state. For example, an order cannot be in a primary state simultaneously. **PAID** and **CANCELLED**. Otherwise, both the operations backend and the user frontend will be in chaos, and in the end, no one will know whether the order is alive, dead, or just a Schrödinger's cat order in the transaction system.

Second, state changes should be triggered by explicit events.

For example, successful payment triggers **PENDING_PAYMENT** → **PAID**. Merchant shipment trigger **PAID** → **SHIPPED**. Triggered by user confirming receipt **SHIPPED** → **COMPLETED**.

Third, the time of critical state changes needs to be recorded.

For example:

- Order creation time;
- Payment time;
- Delivery time;
- Completion time;
- Cancellation time;
- Refund application time;
- Refund completion time.

These time fields are used not only for page display, but also for operational analysis, handling user complaints, and financial reconciliation.

Fourth, the reasons for abnormal states need to be recorded.

For example, the reason for cancellation should be recorded when an order is cancelled; the reason for refund should be recorded when a refund is issued; and the reason for failure should also be recorded when a refund fails. This helps product and operations teams analyze the source of problems.

Fifth, the front-end display state and the back-end actual state must be consistent.

Order status needs to be synchronized between the order details page, order list page, operations backend, and payment system. If the frontend still displays "pending payment" but the backend has already canceled the order, the user experience will be extremely poor.

7.9 Summary

In summary, the initial status after a customer submits an order should be: **PENDING_PAYMENT** After successful payment, the status changes to **PAID** The status changed after the seller shipped the goods. **SHIPPED** The status changes after the user confirms receipt or the system automatically confirms receipt. **COMPLETED** If a user's payment times out or they actively cancel an unpaid order, the status should change to... **CANCELLED**.

Regarding refunds, they are usually allowed. **PAID**、**SHIPPED** and **COMPLETED** You can request a refund for an order in the "Refund Status" section, but whether it will be allowed depends on the platform's after-sales rules. The refund process is usually as follows: **REFUNDING** → **REFUNDED**.

The core goal of order status design is to make the transaction process clear, traceable, and controllable. For product managers, understanding order status is not only about understanding business processes, but also the foundation for understanding system rules, exception handling, and user experience.

VIII. Bug and Anomaly Analysis

This section primarily analyzes the inventory anomaly issues reported by the QA team:

Users see that the product inventory is 1 on the product details page, but when submitting an order, the system displays the message "Insufficient inventory".

On the surface, this seems to be a discrepancy between the inventory displayed on the front-end page and the inventory verified by the system when an order is placed. However, in real e-commerce systems, such issues can be caused by a variety of reasons, including caching, concurrent purchases, inconsistent SKU inventory, changes in shopping cart item status, asynchronous API data, and backend inventory verification.

Therefore, when analyzing this problem, we cannot simply judge whether "the front-end display is wrong" or "the back-end interface is wrong." Instead, we should investigate from multiple perspectives, such as user operation path, front-end and back-end interfaces, caching mechanism, SKU inventory, order submission logic, and database inventory records.

8.1 Possible reason 1: The product details page has a cached inventory, and the displayed inventory is not real-time.

Product detail pages typically see high traffic, so to improve page loading speed, the system may cache product information. For example, product name, price, images, sales volume, and inventory may be cached in Redis, CDN, or the front-end local cache.

If the product details page displays cached inventory instead of real-time inventory in the database, the following situation may occur:

When a user opens the product details page, the page displays an inventory of 1.

However, the actual inventory in the database has already been reduced to 0 by other orders;

When a user submits an order, the backend uses real-time inventory for verification;

The backend detected insufficient inventory and returned a response. “Insufficient inventory”。

In this situation, it is reasonable for the product details page to display outdated inventory data, and for the order interface to return insufficient inventory.

Investigation methods

1. It's necessary to verify the data source for the inventory field on the product details page. For example, check whether the API called by the product details page reads data directly from the database or from cached data.
2. The cache update mechanism needs to be checked. For example, will the cache be updated or expired promptly after inventory changes? In particular, when inventory changes from 1 to 0, will the cache still retain the old value?
3. We need to check the inventory change records when the problem occurred. If the database inventory is already 0, but the cache is still 1, we can determine that the inconsistency in display is caused by the cache not being refreshed in time.

Solution suggestions

For highly real-time fields like product inventory, it's recommended to avoid relying entirely on long-term caching. The following optimization methods can be used:

- Basic product information can be cached, but inventory information should be queried in real time whenever possible;
 - If inventory needs to be cached, a shorter expiration time should be set;
 - Update or delete the cache promptly after inventory deduction;
 - Display prompts on the product details page, such as...["Inventory is based on the verification results at the time of order submission."](#)。
-

8.2 Possible reason two: After the user browsed the product, other users had already placed an order.

In e-commerce systems, inventory is a shared resource. Just because a user sees an inventory level of 1 doesn't mean that inventory will be held for them indefinitely. Unless the system locks the inventory when the user enters the product details page, but most e-commerce systems don't do this because it would result in inventory being wasted on numerous invalid views.

Possible scenarios are:

- User A opens the product details page and sees that the inventory is 1;
- User B purchased the last item before User A submitted their order;
- User B's order has been successfully deducted from the inventory;
- When User A submitted the order again, the inventory had already become 0;
- The system prompted user A ["Insufficient inventory"](#).

This is not a system bug, but a normal scenario of concurrent purchases. Especially when there is only one item left in stock, multiple users purchasing at the same time can easily lead to inventory competition.

Investigation methods

You need to check the order records and inventory flow for this product or SKU during the period when the problem occurred.

Key areas of inspection:

- The time when the user sees the inventory as 1;
- The time when the user submits the order;
- Between these two times, were there any other users who successfully submitted orders or completed payments?
- Did the inventory of this SKU change from 1 to 0 during this period?
- Does the inventory deduction occur during the order submission stage or the payment success stage?

If other orders have already successfully used up the last available inventory before the user submits their order, then this issue falls under normal business logic and is not a system error.

Solution suggestions

The front-end can display a notification when inventory is low, for example:

["Stock is limited, please place your order as soon as possible."](#)

Additionally, the page should provide a more user-friendly message after an order submission fails:

["Product inventory has changed. Please select a different product or refresh the page."](#)

Instead of just showing coldness “Insufficient inventory” Users are not backend log readers, although sometimes it would be nice if they were.

8.3 Possible Reason 3: Inconsistency between SKU inventory and total product inventory

The product details page may display the total inventory of the product. `product stock` However, when submitting an order, the system verifies the inventory of the specific SKU selected by the user. `sku stock`.

For example:

- The total inventory of goods is displayed as 1;
- However, the Black SKU selected by the user has 0 units in stock;
- Another White SKU has 1 in stock;
- If the page does not update the inventory correctly based on the SKU selected by the user, it may still display a total inventory of 1.
- When an order is submitted, the backend checks the inventory based on the Black SKU selected by the user and finds that the inventory is 0.
- The system then indicated that the inventory was insufficient.

This problem is very common with multi-specification products. The total product inventory and SKU inventory must be consistent; otherwise, users will see "product in stock" when the specific specification they selected is actually out of stock.

Investigation methods

We need to check the inventory display logic on the product details page and the order submission parameters.

Key points to note:

1. Does the inventory field displayed on the product details page show the total inventory of the product, or the inventory of the currently selected SKU?
2. After a user selects an SKU, does the page dynamically update the inventory?
3. What is passed when submitting the order? `productId` still `skuId`;
4. The backend actually verifies either the total product inventory or the SKU inventory;
5. Does the total inventory of goods equal the sum of the inventory of all SKUs?

If the page displays the total inventory, but the order submission verification checks the SKU inventory, it is necessary to further confirm whether the SKU-level inventory is 0.

Solution suggestions

Product detail pages should prioritize displaying the inventory of the SKU currently selected by the user, rather than only showing the total inventory of the product.

The interaction logic can be designed as follows:

- If the user does not select an SKU, display the total inventory of the product or a prompt. [Please select specifications.](#);
- After a user selects a specific SKU, the inventory display switches to the inventory of that SKU;
- If the SKU has 0 units in stock, disable the purchase button.
- When submitting an order, you must pass the following: [skull](#) The backend is based on [skull](#) Perform inventory verification and deduction.

This can reduce user misunderstandings and avoid the problem of inconsistency between the total product inventory and the SKU inventory display.

8.4 Possible reason four: The items in the shopping cart are expired or the inventory has changed.

If the user submits the order from the shopping cart, the problem may also stem from the shopping cart data not being updated in a timely manner.

For example:

- When the user added the item to their shopping cart, the inventory was 1.
- The product inventory was subsequently reduced to 0 by other orders;
- The shopping cart page still shows old inventory or the product status has not been updated;
- Users submit their orders directly from the shopping cart;
- The backend detected insufficient inventory during re-verification of the inventory.
- So return [“Insufficient inventory”](#).

In addition, the following anomalies may occur with items in the shopping cart:

- The product has been removed from the shelves.
- SKU has been removed;
- The price of the goods has changed;
- SKU inventory becomes 0;
- The product was modified or banned from sale by the operations staff.

If the shopping cart page does not validate the validity of the products in a timely manner, problems will be exposed during the order submission stage.

Investigation methods

It is necessary to confirm whether the user's action entry point is to purchase directly from the product details page or to submit an order through the shopping cart.

If it's a shopping cart, you need to check:

1. Does the shopping cart page re-fetch the latest status of the products?
2. Is the inventory of items in the shopping cart updated in real time?
3. Are expired items marked in the shopping cart?
4. Does the product validity check occur before submitting an order?
5. Is there a long interval between adding the item to the shopping cart and submitting the order?

If a user added an item to their cart a long time ago and then submitted the order directly, it's reasonable for the submission to fail due to inventory changes.

Solution suggestions

The shopping cart page should re-verify the product status when the user enters the page or selects products.

If the product is expired, the following should be displayed: "The product is no longer available and cannot be purchased."

If inventory is insufficient, the following should be displayed: "Insufficient stock. Please adjust the quantity or delete the item."

If the price changes, the following should be displayed: "The price of the product has changed. Please confirm before submitting your order."

This allows problems to be exposed early in the shopping cart stage, rather than failing only after the user clicks to submit the order.

8.5 Possible reason five: The order submission API re-verifies the inventory, resulting in a difference between the displayed results and the actual inventory.

In e-commerce systems, the inventory displayed on the front end can only serve as a reference for users, not as the final basis for their decision. The true determinant of whether an order can be submitted is the inventory verification result from the backend order submission API.

The reason is simple: front-end data may be outdated or tampered with. If the system only trusts the front-end inventory display, users can even submit abnormal orders by modifying request parameters. Humans sometimes have a greater enthusiasm for writing scripts to exploit vulnerabilities than for writing job application letters.

Therefore, the order submission API must be revalidated:

1. Does the product exist?
2. Is the product available for sale?
3. Is the SKU valid?
4. Is the SKU in stock?
5. Did the purchase quantity exceed the inventory?
6. Has the inventory been locked or deducted by other orders?

If the API detects insufficient inventory after re-verifying the inventory upon order submission, the order should be rejected.

Investigation methods

We need to check the logs and return results of the order submission API.

Key points to note:

- Order submission API received `productId` and `sku` Is it correct?
- Submitted purchase quantity `buyCount` Is it correct?
- What is the real-time inventory as retrieved from the backend?
- Does the inventory have a lock-in record?
- Are there other orders using this inventory?
- Backend response “`Insufficient inventory`” Which specific condition is being considered?

If the backend real-time inventory is indeed 0, then the order submission API rejecting the order is the correct behavior. The focus should be on why the frontend inventory display is not updating synchronously.

8.6 Possible Reason Six: The inventory displayed on the front end is not synchronized with the actual inventory on the back end.

The front-end page may not automatically refresh inventory while the user is on the page. When a user opens a product details page, the inventory is 1, but after the user stays on the page for a while, the inventory may have already been purchased by another user. Because the front-end doesn't refresh, the user still sees the old inventory.

In addition, the following problems may also exist:

- The front end used old API data;
- The front end is not monitoring SKU changes;
- The local state of the front end has not been updated;
- The product details API was not re-requested when the page was redirected.
- The inventory field is displaying incorrectly;
- The front end displays the total product inventory, not the SKU inventory.

These factors can all lead to discrepancies between the front-end display and the actual inventory on the back-end.

Investigation methods

You can check for problems by reproducing the user path:

1. Open the product details page;
2. Record the inventory field returned by the API;
3. Stay for a period of time;
4. Have another user purchase the product;
5. Check if the original page has updated its inventory.
6. Submit the order again to check if there is a shortage of stock.

Additionally, you need to use the browser developer tools to check:

- What is the inventory level returned by the product details API?
- Does the inventory displayed on the front-end page match the API?
- Should the inventory be re-rendered after a user switches SKUs?
- Are the SKU and quantity passed to the order submission interface correct?

If the API returns an inventory of 0, but the page still displays 1, it may be a front-end status update or page rendering issue.

If the API returns an inventory value of 1, but the actual inventory value in the database is 0, the problem may be related to the backend API or caching.

8.7 Recommended troubleshooting steps

To troubleshoot this issue, you can follow these steps:

Step 1: Confirm user operation path

First, we need to confirm where the user entered the order process from:

1. Click "Buy Now" on the product details page;
2. Select items in your shopping cart and submit your order.
3. Submit again after returning to the history page;
4. Access the product page from the event page or external links.

The causes of problems differ depending on the path taken. If it's the shopping cart path, you need to focus on checking the status of the items in the shopping cart; if it's the product details page path, you need to focus on checking the inventory display and SKU linkage on the details page.

Step 2: Confirm the SKU selected by the user and the quantity purchased.

We need to view the request parameters when the user submits the order, including:

- `productId`;
- `sku`;
- `buyCount`;
- User ID;
- Submission time;
- The front-end page displays the inventory.

If the user selects an SKU with 0 inventory, but the total product inventory is 1, the problem is likely that the SKU inventory is displayed incorrectly.

Step 3: Check the return value of the product details page API.

View the actual inventory value returned by the product details API when a user opens the product details page.

Confirmation required:

1. API return **stock** Is it total product inventory or SKU inventory?
2. **skuList** What is the inventory level for each SKU?
3. Does the inventory displayed on the front end match the inventory returned by the API?
4. Does the interface read from the cache?

Step 4: Check the inventory verification results from the order submission API.

Check the order submission API logs to confirm why the backend returned a response. “**Insufficient inventory**”.

Confirmation required:

1. What is the real-time inventory as retrieved from the backend?
2. Has the inventory been locked by other orders?
3. Did the purchase quantity exceed the inventory?
4. Is the product or SKU expired?
5. Are there concurrent orders causing inventory to be preempted?

Step 5: Check inventory flow and order records

View the inventory flow and order records for this SKU during the period when the problem occurred.

Key points to confirm:

1. The time it takes for inventory to go from 1 to 0;
2. Who triggered the inventory deduction?
3. Which order corresponds to the inventory deduction?
4. Did other orders already occupy the last available inventory before the user submitted their order?
5. Are there any failed inventory rollbacks or duplicate deductions?

Inventory transaction records are crucial evidence for investigating these types of issues. Without them, the investigation becomes extremely difficult, akin to finding bugs in an unmonitored system – a true challenge for both product and development teams.

Step 6: Determine the type of problem

Finally, the category of the problem is determined based on the evidence:

- If the database inventory is 0, but the details page still displays 1, it may be due to a caching issue or the front end not refreshing.
- If the total inventory of a product is 1, but the inventory of the SKU selected by the user is 0, it may be a problem with the SKU inventory display logic.

- If other users purchase the last remaining stock after a user opens the page, this is considered a normal concurrent scenario.
 - If the backend inventory verification fails, the development team needs to fix the inventory judgment logic.
 - If the status of items in the shopping cart is not updated, the shopping cart validity validation needs to be optimized.
-

8.8 Product-level optimization suggestions

In addition to technical troubleshooting, product design can also be optimized to address this issue.

1. Retrieve inventory before submitting an order.

Before the user clicks "Submit Order," the front-end can call an API to reconfirm product inventory, product status, and SKU status. If insufficient inventory is found, the user should be notified in advance, rather than an error message appearing after the order is submitted.

2. Display a low inventory notice on the product details page.

When inventory is low, for example, if inventory is less than or equal to 5, a prompt can be displayed: "Stock is limited, please place your order as soon as possible."

This makes users aware that inventory may change at any time, reducing users' misunderstanding of the fixed nature of inventory.

3. Optimize error message text

Don't just show "Insufficient inventory" It can be changed to more user-friendly copy:

"Product inventory has changed. Please select a different product or refresh the page."

If you know the specific reason, you can provide further hints:

"The current specification is sold out. Please select another specification."

4. Differentiate between total product inventory and SKU inventory.

The page should clearly display inventory based on the SKU selected by the user. If the user has not yet selected an SKU, a prompt can be displayed:

Please select product specifications.

After a user selects an SKU, the inventory for that SKU is then displayed. This reduces confusion between total product inventory and SKU inventory.

5. Real-time verification of shopping cart item status

The shopping cart page should verify the status of products before users enter the page, select items, modify quantities, and submit orders.

If a product is removed from the shelves, an SKU becomes invalid, there is insufficient stock, or the price changes, users should be notified promptly for appropriate action.

6. The backend serves as the final source of verification.

Regardless of what the front-end displays, the back-end order submission interface must serve as the final source of inventory verification. When an order is submitted, the product status, SKU status, inventory quantity, and purchase quantity must be re-verified to prevent overselling and abnormal orders.

8.9 Summary

In summary, users see an inventory of 1 on the product details page, but receive a message when submitting an order. “Insufficient inventory” This could be caused by a variety of reasons, including outdated inventory cache on the product details page, other users placing orders before the deadline, discrepancies between SKU inventory and total product inventory, expired product status in the shopping cart, the order submission API re-verifying inventory, and asynchronous inventory data between the front-end and back-end.

When troubleshooting this issue, the analysis should proceed step by step, following the user's operation path, API returned data, SKU inventory, order submission API, inventory flow, and order records. The core principle is: the inventory displayed on the front end should only be used as a reference; real-time inventory verification on the back end is the final basis.

These types of issues require not only identifying the technical causes but also optimizing the user experience. The system should strive to detect inventory changes before users submit orders and provide clear and user-friendly prompts to explain the reasons, reducing user confusion and the negative experience caused by failed transactions.

IX. Design of Core Data Indicators

This section primarily designs the core data metrics that need to be continuously monitored after the e-commerce system goes live. For a basic e-commerce MVP system, the role of data metrics is not only to observe the platform's sales performance, but more importantly, to help product managers determine whether users can successfully complete the core transaction process:

Browse products → Add to cart / Buy now → Submit order → Complete payment → Seller ships → Order complete

Therefore, the design of metrics needs to revolve around user conversion, transaction efficiency, payment stability, product performance, inventory management, and after-sales issues. The following eight core metrics will be the key monitoring targets after the MVP is launched.

9.1 Core Indicator Table

Indicator Name	Indicator Meaning	Calculation method	importance
Product detail page views	Number of times users view product detail page	Product details page PV / UV	Measuring product exposure and user browsing interest
Add-to-cart rate	Percentage of users who added products to their cart after browsing them	Number of users adding items to cart / Number of users visiting the product details page	Measuring product appeal and user purchase intention
Order submission conversion rate	Percentage of users who successfully submitted an order after browsing the products	Number of users submitting orders / Number of users visiting the product details page	Measure the conversion rate from browsing to placing an order.
Payment success rate	The percentage of orders that successfully completed payment after submission.	Number of successfully paid orders / Number of orders submitted	Measuring the smoothness of the payment process and the stability of the payment system
GMV	Total transaction amount of paid orders	Total amount of paid orders	Measuring the overall transaction volume of the platform

Order completion rate	The percentage of completed orders out of paid orders	Number of completed orders / Number of paid orders	Measuring order fulfillment and transaction completion status
Cancellation rate	The percentage of cancelled orders out of submitted orders	Number of cancelled orders / Number of submitted orders	Help identify issues such as pricing, inventory, payment timeouts, or user hesitation.
Refund rate	The percentage of refunded orders out of paid orders	Number of refunded orders / Number of paid orders	Measuring product quality, after-sales experience, and user satisfaction

9.2 Product detail page views

Product detail page views refer to the number of times users visit a product detail page, and can usually be counted in two ways: PV and UV.

- PV represents the number of times a page is viewed.
- UV represents the number of unique users who visit the page.

This metric helps product and operations teams determine whether a product is receiving sufficient exposure. Low product detail page views may indicate an unreasonable product listing page layout, insufficient search result exposure, unclear category entry points, or unattractive product images and titles.

For example, if a product receives no clicks on a listing page for an extended period, it might not be due to a poor product itself, but rather a problem with the product's main image, title, or sorting position. This metric can be used to further analyze the product's exposure path.

The importance of this metric lies in the fact that it's the starting point of the subsequent conversion funnel. Without sufficient visits to the product detail page, subsequent add-to-cart, order placement, and payment are impossible. E-commerce conversion is like a job search; if no one even looks at your resume at the first step, no matter how excellent you are, you'll just languish in the system.

9.3 Add-to-cart rate

Add-to-cart rate refers to the percentage of users who add a product to their shopping cart after browsing the product details page.

The calculation method is as follows:

Add-to-cart rate = Number of users adding to cart / Number of users visiting the product details page

Add-to-cart rate reflects a product's appeal to users. If a product detail page receives many views but has a low add-to-cart rate, it means that although users clicked on the page, they did not develop a further interest in purchasing.

Possible reasons include:

- The product price is not competitive;
- The product images or descriptions are not clear enough;
- Inappropriate inventory or specification selection;
- Insufficient product reviews, after-sales policies, or delivery information;
- The purchase button on the page is not prominent;
- Product detail page loads slowly or provides a poor user experience.

For the MVP stage, the add-to-cart rate is very important because it helps product managers determine whether the product detail page is effectively driving users to the purchase decision stage.

If the add-to-cart rate is low, we can optimize the product main image, product details text, price display, SKU selection experience, and the location of the purchase button.

9.4 Order Submission Conversion Rate

Order submission conversion rate refers to the percentage of users who successfully submit an order after browsing the product details page.

The calculation method is as follows:

Order submission conversion rate = Number of users submitting orders / Number of users visiting the product details page

This metric measures whether users can successfully progress from browsing products to placing an order. Compared to add-to-cart rate, order submission conversion rate is closer to the actual purchase intention. If the add-to-cart rate is high but the order submission conversion rate is low, it may indicate that users are already interested in purchasing, but encountered obstacles in the shopping cart or order submission process.

Possible reasons include:

- Shopping cart operations are complicated;
- The order submission page has too many fields;
- The process of filling in the delivery address is not smooth;

- Insufficient inventory;
- Prices or shipping costs may change on the checkout page;
- The login process is interrupted for the user;
- The page loads slowly or an error occurs.

For product managers, this metric can help pinpoint problems in the conversion funnel. If a large number of users are stuck at the add-to-cart stage but don't submit their orders, the shopping cart and checkout page experiences need to be thoroughly examined.

9.5 Payment success rate

Payment success rate refers to the percentage of orders that are successfully paid for after a user submits an order.

The calculation method is as follows:

Payment success rate = Number of successfully paid orders / Number of submitted orders

This is a crucial metric in e-commerce systems because submitting an order does not guarantee the platform receives transaction revenue. Only after successful payment does the order proceed to the subsequent shipping and fulfillment stages.

A low payment success rate may indicate a problem with the payment process.

Common causes include:

- Payment interface error;
- Unstable payment channels;
- User payment timed out;
- Payment page redirection failed;
- The payment method selection is unclear;
- Order amount is abnormal;
- Users change their purchase decisions before making a payment.

The MVP phase requires particular attention to monitoring payment success rates, as the payment process is a critical node in the core transaction loop. If users submit their orders but are lost in large numbers at the payment stage, it's like sending users to the checkout counter only to have the cash register go on strike – the business system instantly loses its dignity.

9.6 GMV

GMV is an abbreviation for Gross Merchandise Volume, which refers to the total transaction amount of orders paid by the platform within a certain period of time.

The calculation method is as follows:

GMV = Total amount of paid orders

In this project, it is recommended to only count the amount of paid orders, not the amount of all submitted orders. This is because unpaid orders do not actually constitute transaction value, and including unpaid orders in GMV would lead to inflated data.

GMV can measure the overall transaction volume of a platform and is one of the most basic operating indicators for e-commerce businesses.

Through GMV, product and operations teams can observe:

- Overall sales trends on the platform;
- Sales changes over different time periods;
- Sales contribution of different products;
- The impact of activities or promotions on transaction volume;
- Changes in user purchasing power and average order value.

However, it's important to note that GMV alone cannot represent the health of a business. A system may have a high GMV, but also a high refund rate or a low payment success rate. Therefore, GMV needs to be analyzed in conjunction with other metrics such as order completion rate, refund rate, and payment success rate.

9.7 Order completion rate

Order completion rate refers to the percentage of orders that have reached the completed status out of all paid orders.

The calculation method is as follows:

Order completion rate = Number of completed orders / Number of paid orders

This metric is used to measure the success of order fulfillment. After a payment is successful, an order still needs to go through the processes of merchant shipping, logistics delivery, user receipt or automatic system confirmation of receipt before it will finally enter the "completed" status.

A low order completion rate may indicate a problem with the fulfillment process.

Common causes include:

- The seller did not ship the goods on time;
- Insufficient inventory prevented shipment.
- Logistics abnormality;
- The user has not confirmed receipt for an extended period of time;
- The order was refunded or cancelled;
- The efficiency of back-end order processing is low.

For an MVP system, order completion rate helps product managers determine if the transaction loop is truly working. Because an e-commerce system isn't just about getting users to pay; it's also about getting them to receive the goods. Otherwise, it's not e-commerce; it's a wishing well, and a wishing well with a payment gateway at that.

9.8 Order cancellation rate

The order cancellation rate refers to the percentage of cancelled orders out of the total number of orders submitted.

The calculation method is as follows:

Cancellation rate = Number of cancelled orders / Number of submitted orders

Order cancellation includes user-initiated cancellation and automatic system cancellation, such as automatic cancellation due to payment timeout.

This metric can help product managers analyze why users did not complete a transaction.

A high cancellation rate may indicate the following:

- The user hesitated after placing the order;
- The payment time limit is unreasonable;
- The prices of the goods are unattractive;
- The shipping cost or discount information is unclear;
- Inventory changes led to order cancellations;
- After submitting the order, you discovered that there were problems with the address, specifications, or amount.
- Unsatisfactory payment process leads users to abandon the service.

It is recommended to further differentiate the reasons for cancellation, for example:

- ☐ User canceled;

- ☐ Payment timed out and cancelled;
- ☐ Cancellation due to insufficient stock;
- ☐ Cancel in the background;
- ☐ Risk control measures have been cancelled.

This helps the team pinpoint the problem more accurately, rather than just looking at the total cancellation rate. The total cancellation rate only tells us "something went wrong," but it doesn't tell us "who did it." Looking at this kind of data is pointless and only makes meetings longer.

9.9% refund rate

The refund rate refers to the proportion of refunded orders to the total paid orders.

The calculation method is as follows:

Refund rate = Number of refunded orders / Number of paid orders

Refund rate can reflect product quality, user satisfaction, after-sales experience, and the accuracy of product information.

A high refund rate may indicate the following:

- The product description does not match the actual product;
- There are problems with the quality of the goods;
- User made a mistaken purchase;
- The logistics time is too long;
- The after-sales policy is unclear;
- The user changed their mind after purchasing;
- There are concentrated complaints about certain products.

For product managers, the refund rate is not only an after-sales metric, but it can also help optimize product detail pages, SKU descriptions, image displays, and purchase prompts.

For example, if a product's refund rate is significantly higher than others, further analysis of the reasons for refunds is needed. It could be due to a quality issue with the product itself, or the product details page might be overly exaggerated, creating unrealistic expectations for users. Humans are easily lured by attractive main images, only to be misled by reality upon receiving the actual product; this is one of the reasons why refund rates exist.

9.10 Indicator Funnel Analysis

In addition to monitoring each metric individually, these metrics should be combined into a complete transaction conversion funnel.

A basic e-commerce conversion funnel can be designed as follows:



Figure 9-10 Core Conversion Funnel Diagram

By using funnel analysis, product managers can determine at which stage users primarily churn.

- If the product details page receives few views, it indicates insufficient exposure.

- If a product has high page views but a low add-to-cart rate, it indicates that the product is not attractive enough or the product details page is not sufficiently informative.
- If the add-to-cart rate is high but the order submission conversion rate is low, it indicates a problem with the shopping cart or the order submission process.
- If the order submission conversion rate is high but the payment success rate is low, it indicates that there may be a problem with the payment process.
- If the payment success rate is high but the order completion rate is low, it indicates that there may be problems in the fulfillment or logistics process.

This funnel analysis is more valuable than looking at GMV alone because it helps the team pinpoint the specific stages where the problem occurs.

9.11 Data Tracking Recommendations for the MVP Phase

To support the above metric statistics, basic tracking points need to be designed in advance during the MVP phase. It is recommended to record at least the following user behavior events:

- Product detail page exposure incident: `product_detail_view`
- Add to cart event: `add_to_cart`
- Instant Purchase Event: `buy_now_click`
- Order submission event: `submit_order`
- Payment success event: `payment_success`
- Payment failure event: `payment_failed`
- Order cancellation events: `cancel_order`
- Refund request incident: `request_refund`
- Order completion event: `order_completed`

It is recommended to record basic fields for each event, such as:

- User ID;
- Product ID;
- SKU ID;
- Order ID;
- Time of the event;
- Page source;
- Commodity prices;

- Purchase quantity;
- Order status;
- Reason for failure or cancellation.

These tracking points allow for subsequent analysis of user behavior paths, product conversion rates, reasons for payment anomalies, and order fulfillment status.

9.12 Summary

In summary, after the MVP of an e-commerce system is launched, the following eight core metrics should be monitored: product detail page views, add-to-cart rate, order submission conversion rate, payment success rate, GMV, order completion rate, order cancellation rate, and refund rate.

These metrics cover the entire journey from a user browsing products to completing a transaction, helping product managers determine if the system is truly functioning correctly in its core transaction process.

Furthermore, combining funnel analysis and event tracking can further pinpoint the reasons for user churn and system anomalies.

For the MVP stage, the data metrics don't need to be overly complex from the start, but they must be able to answer a few key questions: Did the user see the product? Did the user intend to buy it? Could the user place an order successfully? Was the payment successful? Was the order completed? And how serious were the after-sales issues? As long as these questions can be answered by data, product managers can conduct subsequent iterations based on real user behavior, rather than relying on imagination in a meeting room to guide the system's evolution.

10. Summary and Future Iteration Directions

This solution is designed around an MVP version of a basic e-commerce system. Its core objective is to prioritize completing the basic transaction loop from a user browsing products to completing an order, namely:

Browse products → Add to cart / Buy now → Submit order → Pay → Ship → Order complete

In the product design section, this solution first clarifies the main user roles in the system, including customers/buyers, operations personnel/merchants, and an optional platform administrator. Different roles have different core objectives within the system: customers primarily focus on browsing, purchasing, paying, and viewing orders; operations personnel primarily focus on product, inventory, order, and shipping management; and platform administrators are more concerned with permissions, abnormal orders, and the overall system operation.

In the MVP feature planning, this solution prioritizes features according to P0, P1, and P2. P0 features primarily revolve around the core transaction process, such as product listings, product detail pages, shopping carts, order submission, payment, order management, product management, and inventory management. P1 features mainly improve user experience and operational efficiency, such as search filtering, order cancellation, refund processing, and data dashboards. P2 features can be used for subsequent version iterations, such as coupons, recommendation systems, rating systems, and more complex marketing tools.

In the process design section, this solution focuses on outlining the customer order placement and payment process, as well as the product creation and publishing process. For e-commerce systems, process design must not only cover the normal path but also consider abnormal situations, such as insufficient inventory, payment failure, payment timeout, and products remaining in the shopping cart even after being removed from the platform. These abnormal scenarios directly impact user experience and system stability, therefore they need to be clearly designed in advance in product rules, page prompts, and backend validation.

In the PRD section, this solution uses a specific function as an example to illustrate the functional background, user objectives, page fields, business rules, exception handling, and acceptance criteria. The simplified PRD focuses not on complexity, but on making the requirements clear enough that designers, developers, testers, and operations personnel can understand how the function should be implemented, how to determine completion, and how the system should respond in case of exceptions.

In the technical understanding section, this solution illustrates the basic technical understanding skills that product managers need through API responses, pseudocode logic, and order status transitions. Product managers don't necessarily need to write complete code, but they do need to understand the meaning of API fields, price units, product status, SKU inventory relationships, order state machines, and basic

business validation logic. Only by understanding these aspects can they communicate more accurately with development and avoid making impractical or poorly defined requirements.

In the anomaly analysis section, this solution analyzes the possible causes of "the product details page displays an inventory of 1, but an insufficient inventory message appears when submitting an order." This issue may be caused by caching, concurrent purchases, discrepancies between SKU inventory and total product inventory, expired items in the shopping cart, the order submission API re-verifying inventory, and data asynchrony between the front-end and back-end. Troubleshooting these types of issues requires a step-by-step approach, examining user paths, API data, SKU inventory, order logs, and inventory flow records. The core principle is: the front-end display can serve as a user reference, but real-time back-end verification must be the final basis for judgment.

In terms of data metrics, this solution designs core metrics that need to be monitored after the e-commerce system goes live, including product detail page views, add-to-cart rate, order submission conversion rate, payment success rate, GMV, order completion rate, order cancellation rate, and refund rate. These metrics cover the entire chain from product exposure to transaction completion, helping product managers determine whether the system has truly achieved a smooth transaction loop and identify at which stage user churn and system anomalies occur.

10.1 The Core Value of the MVP Phase

The core value of this MVP version lies in validating the basic transaction capabilities of an e-commerce system using minimum viable features. For customers, the MVP version should ensure that users can smoothly complete operations such as browsing products, selecting specifications, adding items to the cart, submitting orders, making payments, and checking order status. For operations personnel, the MVP version should ensure that the backend can complete product creation, inventory maintenance, order viewing, and shipment management. For the platform, the MVP version should ensure that a stable data loop is formed between product, inventory, order, payment, and logistics status. Therefore, the MVP stage should not excessively pursue complex marketing functions, but should prioritize ensuring the stability, clear status, accurate inventory, and reliable payment of the core transaction links. Otherwise, no matter how beautifully designed the coupons, recommendation systems, and complex activities are, if users ultimately cannot make purchases, it is just dressing the system in an expensive but useless coat.

10.2 Future Iteration Directions

After the MVP version is launched and the basic transaction process is verified, the system can be iterated in the following directions.

1. Improve the user purchase experience

Further optimizations can be made to features such as product search, category filtering, product recommendations, product reviews, favorites, coupons, and promotional activities. These features will help users find products faster and improve purchase conversion rates.

For example, adding user reviews and Q&A to the product details page can enhance user trust; adding filters and sorting to the product list page can improve search efficiency; and adding coupons to the checkout page can increase users' willingness to place an order.

2. Optimize inventory and order management capabilities

The MVP stage can initially support basic inventory management, but subsequent stages should further support inventory alerts, inventory flow tracking, inventory locking, batch inventory adjustments, and inventory anomaly alerts.

In terms of order management, features such as order filtering, bulk shipping, logistics tracking, abnormal order marking, and after-sales processing records can be further added to improve the processing efficiency of operations personnel.

3. Improve after-sales and refund processes

The MVP stage can initially support basic order cancellation and refund status, but subsequent stages should further improve features such as refund reasons, refund review, return logistics, after-sales time limits, partial refunds, and refund failure handling.

After-sales service directly impacts user satisfaction and long-term trust in the platform. If the refund process is unclear, users can easily go from wanting to buy something to wanting to complain about the platform – such is the fragile yet rational nature of human consumer psychology.

4. Establish a more complete data analysis system

Going forward, we can build a data dashboard and funnel analysis system based on the core metrics from the MVP stage.

For example, we can analyze:

- Page views and conversion rates for different products;
- User conversion rates across different channels;
- Sales performance of different SKUs;
- Distribution of reasons for payment failure;
- Distribution of reasons for order cancellation;
- Distribution of refund reasons;
- User repurchase rate.

This data can help product and operations teams continuously optimize products, pages, processes, and marketing strategies.

5. Improve system stability and security

As order volume increases, the system needs to pay closer attention to stability and security. For example:

- Prevent overselling of inventory;
- Ensure accurate payment callbacks;
- Prevent duplicate order submissions;
- Preventing malicious order placement;
- Ensure user data security;
- Ensure that order and payment data are traceable;
- Add anomaly monitoring and alarm mechanisms.

Although these capabilities may not be directly displayed on the page, they determine whether the system can run stably in the long term.

10.3 Final Summary

Overall, this solution is based on the MVP of an e-commerce system and has been fully designed around user roles, functional priorities, core processes, PRD, API understanding, code logic, order status, exception analysis, and data metrics.

The focus of this solution is not on designing a complex, large-scale e-commerce platform all at once, but rather on ensuring the stable operation of the basic transaction loop. For the MVP phase, the most

important thing is to enable users to buy products, enable operations staff to manage products and orders, and enable the system to accurately handle inventory, payment, delivery, and order status.

Subsequent versions can further expand upon the MVP to include search recommendations, marketing tools, after-sales support, data analysis, and risk control capabilities. In this way, the system can first complete core validation from scratch, and then gradually move from basic usability to experience optimization and operational growth.

Therefore, the core idea of this solution can be summarized as follows:

1. First, ensure the transaction loop runs smoothly, then optimize the user experience;
2. First ensure accurate inventory and orders, then expand marketing capabilities;
3. First, validate the basic value through an MVP, and then drive subsequent iterations with data.